

Universal, Hard-Routed and Soft Mixture-of-Experts Image Restoration and Style-Conditioned Face-to-Sketch Synthesis: From Optuna-Tuned Models to a Containerised Web Application

Zaid Fareed

Roll No. 23I-0829, Section C — Generative AI, Assignment 1

Source code: https://github.com/Zaid-fareed/23i-0829_C_GenAi_A1

Demonstration video: [\[paste the YouTube link here\]](#)

Abstract—We design, train, evaluate and deploy four related generative models. The first three restore images corrupted by salt-and-pepper noise, Gaussian blur or rectangular occlusion on the Oxford-IIIT Pet dataset: (1) one universal denoising autoencoder with a genuine bottleneck, (2) a hard-routed system in which a corruption classifier selects one of three specialist autoencoders, and (3) a jointly fine-tuned soft mixture-of-experts whose gate assigns a continuous weight to an identity branch and the three experts. The fourth model is a style-conditioned conditional GAN (U-Net generator, PatchGAN discriminator) that turns FS2K face photographs into sketches in one of three styles. All hyper-parameters were searched with Optuna, all runs are tracked with MLflow, and all inference models are exported to ONNX and verified against PyTorch (maximum difference below 3×10^{-6}). Tested on 36,690 deterministic inputs (every test image under all ten conditions), the universal autoencoder raises the mean SSIM of corrupted inputs from 0.636 to 0.767, but reconstructs clean images at only 0.814 (an identity mapping would score 1.0), which motivates routing. The corruption classifier is 0.9941 accurate; hard routing reaches an overall SSIM of 0.779 and the soft mixture 0.783 (universal: 0.772) while keeping clean images almost unchanged (0.996 and 0.991); the gain comes from clean inputs, and on corrupted inputs the three systems are within about 0.02 SSIM of each other. SSIM and PSNR disagree for occlusion, which we analyse. The cGAN produces recognisable but soft sketches (SSIM 0.486 on the official test set). Everything runs from one docker compose up through a FastAPI + React/Tailwind application whose interface was designed in Google Stitch. We report failure cases and two corrections that our own measurements forced: a first autoencoder that under-fitted, and first specialists that lost more than half of the image colour.

Index Terms—denoising autoencoder, mixture of experts, conditional GAN, Optuna, ONNX, FastAPI, image restoration, face sketch synthesis

I. Introduction

Image restoration asks for a clean image x given a degraded observation $\tilde{x}$. A single network must cope with very different degradations: salt-and-pepper noise destroys isolated pixels, Gaussian blur removes high frequencies everywhere, and occlusion removes whole regions for which no information remains. This report studies how the same restoration problem behaves under three architectural

choices and then adds a fourth, paired image-to-image task:

- 1) Task 1, universal denoising autoencoder: one convolutional autoencoder with a real bottleneck that is never told which corruption was applied.
- 2) Task 2, hard routing: a four-class corruption classifier followed by one specialist autoencoder per corruption, with an identity bypass for clean inputs.
- 3) Task 3, soft mixture of experts: a differentiable gate that mixes an identity branch and the three specialists, initialised from Task 2 and fine-tuned jointly.
- 4) Task 4, face-to-sketch cGAN: a U-Net generator and PatchGAN discriminator conditioned on one of three FS2K sketch styles.

Beyond the models, the assignment requires the complete path from research to a working product: Optuna searches for every task, MLflow experiment tracking, ONNX export with numerical verification, and a containerised web application (FastAPI back end, React and Tailwind front end) whose interface was first designed in Google Stitch. The source code, configuration, Optuna studies, export code, Dockerfiles and instructions are in the repository at https://github.com/Zaid-fareed/23i-0829_C_GenAi_A1.

Contributions. (i) A complete, reproducible implementation of the four tasks with a deterministic evaluation protocol in which every test image is evaluated under all ten conditions (36,690 inputs). (ii) A quantitative comparison of universal, hard-routed and soft-mixture restoration, including an analysis of classifier-induced failures and of the gate's behaviour. (iii) Two diagnosed and corrected failures from our own experiments: an under-fitted first autoencoder, and specialists whose SSIM-heavy loss weight made them discard colour. (iv) A deployed application that exposes all models with timing, routing and error-map information, and whose claims are checked by automated tests.

Organisation. Section II reviews the literature and records the design decisions. Section III defines the data, the corruption protocol and the metrics. Sections IV–VII treat the four tasks in turn (architecture, losses, training, Optuna, results, failures). Section IX describes the application and deployment, Section X states limitations, and the appendices contain the AI-use statement and reproduction steps.

II. Related Research and Design Decisions

Denosing autoencoders. Vincent et al. [1], [2] showed that training an autoencoder to reconstruct a clean input from a stochastically corrupted one yields robust representations. Later restoration networks added depth and skip connections: DnCNN learns the residual noise [3] and REDNet connects encoder and decoder symmetrically [4]. Context encoders restore large missing regions by inpainting [5]. Our Task 1 follows the classical bottleneck design because the assignment forbids unrestricted skip connections; the price, quantified in Section IV, is that even clean images are reconstructed imperfectly.

Reconstruction losses. Pixel-wise L1 gives sharper results than L2 but ignores structure; SSIM [6] compares local luminance, contrast and structure. Zhao et al. [7] showed that combining an L1 term with an (MS-)SSIM term trains restoration networks better than either alone. We therefore use $\alpha\mathcal{L}_1 + (1 - \alpha)(1 - \text{SSIM})$ and let Optuna choose α . A caveat that shaped our Task 2 fix: SSIM responds weakly to global colour shifts, so a loss dominated by the SSIM term permits colour loss (Section V-C). Blau and Michaeli [8] explain why pixel and structural metrics reward blur as distortion falls, which is relevant to the softness of our sketches.

Mixture of experts. Jacobs et al. [9] introduced gated combinations of specialised networks trained jointly; sparsely-gated layers scaled the idea to very large models [10] and introduced auxiliary losses that keep expert usage balanced [11]. Our Task 3 uses a dense softmax gate with temperature and the assignment’s squared-deviation balance regulariser, initialised from trained components to avoid starting from random experts.

Conditional image-to-image translation. GANs [12] can be conditioned on side information [13]. Pix2pix [14] combines a U-Net generator [15], a PatchGAN discriminator that judges local patches [16] and an L1 term weighted by $\lambda = 100$. Face sketch synthesis has a long history [17]; FS2K [18] provides 2,104 photo-sketch pairs in three artist styles. Our Task 4 adds a learned style embedding to both the generator and the discriminator.

Tooling. Optuna [19] provides tree-structured Parzen estimator sampling and pruning, MLflow [20] records runs, ONNX Runtime [21] executes the exported models, FastAPI [22] serves them, Docker [23] packages the stack, and Stitch [24] produced the interface design. Optimisation uses AdamW [25] (Adam [26] with decoupled weight decay) with cosine annealing [27]; convolution blocks use

batch normalisation [28]. The datasets are Oxford-IIIT Pet [29] and FS2K [18].

III. Data, Corruption Protocol and Metrics

Oxford-IIIT Pet (Tasks 1–3). The dataset contains 7,393 images of 37 cat and dog breeds [29]. Following the assignment, the official train/validation collection (3,680 images) was split 80/20 with random seed 42 into 2,944 training and 736 validation images; the official 3,669 test images were not used for training or tuning. Category labels are ignored, and all images are converted to RGB and resized directly (bicubic) to 128×128 . The split is stored once in manifests/pets_split.json and reused by all three tasks; the training images are decoded once and kept in memory as uint8 tensors, which is why an epoch takes seconds.

Runtime corruption. No corrupted copy of the data is stored. For every training image that is loaded, one of four conditions is drawn with equal probability and a fresh random specification is sampled (Table II). A specification is a plain record that fully describes the corruption (probability and noise seed, kernel and sigma, or rectangle coordinates); the same pure function `apply_spec` applies it during training, validation and testing, and the FastAPI back end contains a NumPy port that our tests compare against the training code. The classifier’s balanced batches draw exactly $B/4$ items of each class.

Deterministic manifests. A validation manifest (736 entries, exactly 184 per class) and test manifests are generated once and store, for each entry, the image, corruption type, severity, parameters, mask coordinates and random seed, with a SHA-256 digest. Two test manifests exist. The first (3,669 entries) gives every test image one condition and was used by the Kaggle evaluation scripts. The assignment, however, asks that every test image have three fixed severities of every corruption, so the final protocol uses the full manifest: each of the 3,669 test images under all ten conditions, 36,690 inputs. All test tables in this report use the full manifest unless stated otherwise.

FS2K (Task 4). FS2K [18] provides 2,104 photo-sketch pairs in three sketch styles. We use the official training and test definitions; 15% of the official training portion (1,058 pairs) forms a validation set, stratified by style with seed 42, leaving 899 training and 159 validation pairs, and the 1,046 official test pairs (619, 381 and 46 in styles 1, 2 and 3) are touched only for final evaluation. Photographs are resized to 128×128 RGB and sketches to 128×128 grayscale. Augmentation (horizontal flip and a random crop-and-resize) uses one random draw applied identically to both members of a pair.

Metrics. For restoration we report the mean absolute error (L1, images in $[0, 1]$), the structural similarity index SSIM (Gaussian window of 11 pixels, $\sigma = 1.5$) and PSNR in dB. PSNR of identical images is infinite, so values are clipped at 60 dB; means over conditions that contain

TABLE I
Main design decisions, the alternatives considered, and the evidence behind the choice made.

Decision	Alternatives considered	Choice and evidence
Skip connections in the autoencoder	Symmetric or residual skips [3], [4]	None: the assignment requires a genuine bottleneck (latent of 2,048 values, 4.2% of the pixels). Cost measured in Sec. IV: clean images are reconstructed at SSIM 0.814, below the untouched input. Identity bypasses in Tasks 2–3 remove this cost.
L1 versus SSIM weight α	Fixed $\alpha = 0.8$ (assignment default), pure L1, pure SSIM	Searched with Optuna. The first specialist search chose $\alpha = 0.29$ and produced output with about 43% of the original colour; restricting the search to $\alpha \geq 0.55$ restored it to 89% (Sec. V-C).
Routing rule	Hard (argmax) versus soft (softmax with temperature)	Both built and compared on identical inputs (Table IX); the soft gate was trained from the classifier weights with a gate-only warm-up.
Balance regulariser	Entropy or load-balancing losses [11]	The squared deviation from a uniform mean weight of the assignment: differentiable, one weight to tune, and no expert became inactive (Table XI).
Hyper-parameter search	Grid or random search	TPE sampling with median pruning [19] on short, sub-sampled trials; all studies and counts are in Table XVI.
GAN objective	Least-squares, hinge or spectral-normalised discriminators	Non-saturating BCE with logits as specified; other objectives were not explored (stated as a limitation). A second training recipe (photo-only jitter plus learning-rate decay) was tried and rejected (Table XII).
Input framing for sketches	Stretch, pad or centre-crop non-square photos	Crop (default) after a test on a tall portrait showed that stretching distorted the face and degraded the sketch; the app exposes all three options.



Fig. 1. The ten deterministic test conditions for one test image (clean and three fixed severities of each corruption).

identity-routed images should be read with this in mind. As a reference we always report the “no restoration” score of the corrupted input itself, because a restoration model is only useful where it beats that baseline. Classification is evaluated with accuracy, macro-averaged precision, recall and F1, per-class values and a row-normalised confusion matrix.

IV. Task 1: Universal Multi-Corruption Denoising Autoencoder

A. Architecture

The network (Fig. 2) has a convolutional encoder, a compressed latent, and a convolutional decoder, and receives no information about the corruption type. The encoder halves the spatial size four times ($128 \rightarrow 8$) while widening the channels (48, 96, 192, 384); each stage is a stride-2 3×3 convolution followed by a 3×3 convolution, both with batch normalisation and ReLU. A 1×1 convolution projects to a linear latent z of $32 \times 8 \times 8 = 2,048$ values (4.2% of the 49,152 pixel values), followed by Dropout2d. The decoder mirrors the encoder with nearest-neighbour $2 \times$ upsampling and 3×3 convolutions and ends in a 3×3

convolution with a sigmoid. There are no skip connections: every pixel of the output must be generated from z . The model has 4.01 M parameters.

B. Loss and training

With $\hat{x} = D(E(\tilde{x}))$ the loss is

$$\mathcal{L}_{UDAE} = \alpha \mathcal{L}_1(x, \hat{x}) + (1 - \alpha)(1 - \text{SSIM}(x, \hat{x})), \quad (1)$$

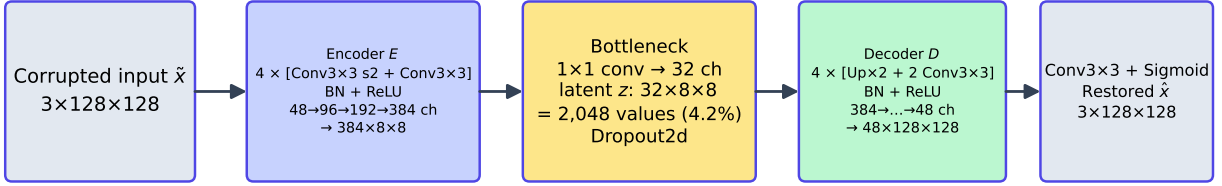
where L1 encourages accurate pixels and SSIM preserves structure and local contrast. Optuna selected $\alpha = 0.58$ (the suggested starting value was 0.8). Training uses AdamW (weight decay 10^{-4}), a cosine learning-rate schedule, gradient clipping at 1.0, mixed precision on an NVIDIA T4, batch 16 and learning rate 4.6×10^{-4} for 100 epochs over all 2,944 training images, each drawn with a fresh random corruption. The checkpoint with the lowest validation objective $0.5 \mathcal{L}_1 + 0.5 (1 - \text{SSIM})$ is kept (epoch 93); this objective does not depend on α and is therefore comparable across trials. The search space, trial counts and selected values are in Section VIII.

TABLE II

Complete corruption configuration. Training values are re-sampled every time an image is loaded; validation uses the random ranges once with fixed seeds; testing uses the three fixed severities.

Condition	Training / validation (random)	Fixed test severities (low, medium, high)
Clean	original image	original image
Salt-and-pepper	$p \sim U(0.02, 0.15)$; exactly $\text{round}(pHW)$ pixels (all channels) set to black or white with equal probability	$p = 0.03, 0.08, 0.15$
Gaussian blur	kernel size from $\{3, 5, 7\}$, $\sigma \sim U(0.5, 2.5)$, separable Gaussian with reflect padding	$(k, \sigma) = (3, 0.7), (5, 1.5), (7, 2.5)$
Rectangular occlusion	1–3 black rectangles, union area between 10% and 35% of the image, random positions and aspect ratios	1, 2, 3 rectangles covering 10%, 20%, 35% ($\pm 1\%$)

Task 1 - universal denoising autoencoder (one model for clean, salt-and-pepper, blur and occlusion inputs)



$$\mathcal{L}_{DAE} = \alpha \mathcal{L}_{L1}(x, \hat{x}) + (1 - \alpha)(1 - \text{SSIM}(x, \hat{x})), \alpha = 0.58 \mid \text{no skip connections: everything passes through the latent } z \mid 4.0 \text{ M parameters}$$

Fig. 2. Universal denoising autoencoder. The only path from input to output goes through the 2,048-value latent.

C. A first attempt that under-fitted

Our first complete run (search space with dropout up to 0.3, latent sizes 16–128, three-epoch trials on 600 images, then 30 epochs on 736 images) was measured on the single-condition test manifest and reached only SSIM 0.545 and PSNR 19.55 dB. Two observations identified the cause. First, the train–validation gap was only 0.008 SSIM: the model was not memorising, it was under-fitting. Second, clean, blurred and noisy inputs all scored about 0.55–0.56 SSIM, so even clean images were reconstructed no better than corrupted ones: the limit was the model’s ability to reconstruct at all. Optuna had chosen dropout 0.29 (dropout in an autoencoder deletes the very features needed for reconstruction) and the largest allowed latent, because three-epoch trials reward configurations that learn fast early. We therefore capped the dropout at 0.15, added latent sizes up to 256, lengthened trials to six epochs, and trained the final model for 100 epochs on all training data. On the same manifest the SSIM rose from 0.545 to 0.779.

D. Results

Table III gives the results on the full test protocol, separated by corruption and by severity, next to the score of the untouched corrupted input (“no restoration”), which any useful restoration must beat. Figure 3 summarises all three restoration systems.

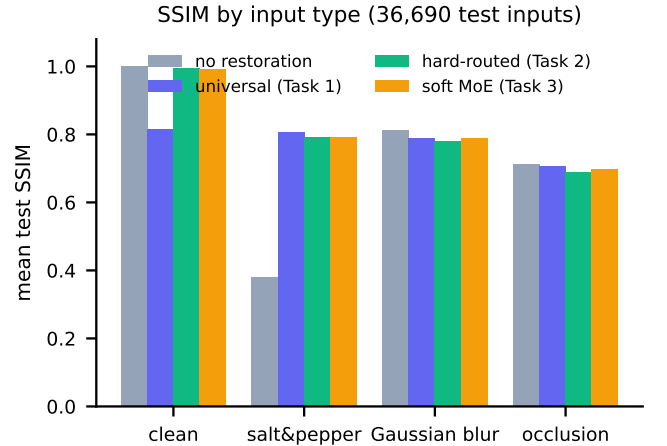


Fig. 3. Mean test SSIM per input type for the corrupted input, the universal autoencoder, hard routing and the soft mixture.

Salt-and-pepper noise is restored well. SSIM rises from 0.382 to 0.805 and PSNR from 16.4 to 25.8 dB; the output quality is almost independent of severity (0.813 to 0.796 SSIM from $p = 0.03$ to $p = 0.15$), because the noise is removed by averaging over the many uncorrupted neighbours.

Blur is not helped, and clean images are damaged.

TABLE III

Task 1 (universal autoencoder) on the 36,690 deterministic test inputs. PSNR is clipped at 60 dB (≥ 60 for the identical clean input).

Corruption	Setting	Level	Input (no restoration)		Universal autoencoder		
			SSIM	PSNR	L1	SSIM	PSNR
clean	—	—	1.000	≥ 60	0.036	0.814 ± 0.080	25.97
salt-and-pepper	$p=0.03$	low	0.602	20.13	0.036	0.813 ± 0.080	25.95
salt-and-pepper	$p=0.08$	medium	0.339	15.87	0.036	0.808 ± 0.081	25.89
salt-and-pepper	$p=0.15$	high	0.204	13.14	0.037	0.796 ± 0.082	25.68
Gaussian blur	$k=3, \sigma=0.7$	low	0.944	32.34	0.035	0.814 ± 0.081	26.17
Gaussian blur	$k=5, \sigma=1.5$	medium	0.806	26.77	0.036	0.805 ± 0.083	26.04
Gaussian blur	$k=7, \sigma=2.5$	high	0.692	24.35	0.039	0.751 ± 0.097	25.13
occlusion	1 rect, 10%	low	0.861	16.67	0.043	0.768 ± 0.083	23.56
occlusion	2 rects, 20%	medium	0.730	13.34	0.051	0.718 ± 0.086	21.66
occlusion	3 rects, 35%	high	0.547	10.77	0.067	0.634 ± 0.091	19.21
all corrupted (33,021)			0.636	19.27	0.042	0.767	24.36
all conditions (36,690)			0.672	23.34	0.042	0.772	24.52

For Gaussian blur the output scores 0.790 SSIM and 25.8dB while the untouched input already scores 0.814 and 27.8dB; at $k=5$ the two tie (0.805 against 0.806) and only for the strongest blur ($k=7$) does restoration beat the input in SSIM (0.751 against 0.692). Clean images fall from 1.000 to 0.814. The bottleneck cannot reproduce fine detail, so mild blur is “restored” into a slightly different soft image. This is the central limitation of the universal design and the motivation for Tasks 2 and 3: an identity path costs nothing on inputs that do not need restoration.

Occlusion: SSIM and PSNR disagree. Overall the output scores 0.707 SSIM against 0.713 for the masked input, yet PSNR rises from 13.6 to 21.5dB. SSIM is computed in local windows, so the large part of the image outside the black rectangles is already perfect and the mean SSIM of the untouched input stays high; doing nothing scores a higher SSIM than the autoencoder on 87% of the low-severity (10%), 52% of the medium and 19% of the high-severity inputs. PSNR and visual inspection (Fig. 5) show what SSIM hides: the autoencoder replaces the black holes by plausible content, which removes the large pixel errors inside the mask. Only at 35% coverage does restoration also win in SSIM (0.634 against 0.547). We therefore read both measures together.

Generalisation. On 736 training images corrupted with validation-style fixed corruptions the SSIM is 0.797, against 0.783 on the 736 unseen validation images (gap 0.014); training and validation curves (Fig. 4) move together, so the model neither memorises nor over-fits.

Failure cases. Figure 6 shows the four worst corrupted inputs; all are 35% occlusions. In the first, a cat on a black background, the mask merges with the background and cuts the animal in two; the network reconstructs a smeared grey-brown shape (SSIM 0.27). In the second, the three rectangles cover the head and back of a dog on grass; the output is a blurred brown-green patch without the animal (0.31). In the third, an L-shaped mask removes the head of a basset hound and the network paints wall

and grass texture instead (0.34). In the fourth, two masks cover the body of a brown dog and a third one the top right (0.35). In all four the error is concentrated inside the masks and along the animal’s outline: the information is absent, so the best the autoencoder can do is a smooth guess that matches the surroundings; it cannot reconstruct the subject.

V. Task 2: Corruption Classification and Hard-Routed Specialists

A. Corruption classifier

Architecture and training. The classifier maps $\tilde{x}$ to $p = [p_{\text{clean}}, p_{\text{salt}}, p_{\text{blur}}, p_{\text{occ}}]$. It has four stride-2 stages of two 3×3 convolutions (16, 32, 64, 128 channels, batch norm, ReLU, light Dropout2d), global average pooling, dropout and a linear layer; it has only 0.295 M parameters. The predicted class is $r = \arg \max_k p_k$. Training minimises the multiclass cross-entropy, and every batch is exactly balanced: a batch of size B contains $B/4$ clean, salt-and-pepper, blur and occlusion items, with labels taken from the runtime corruption pipeline. Optuna tuned the learning rate, batch size, channel configuration, dropout and weight decay (objective: validation macro-F1); the selected values are learning rate 1.5×10^{-3} , batch 32, dropout 0.15 and weight decay 1.3×10^{-4} , trained for 30 epochs.

Results. On the full test protocol (36,690 inputs) the classifier has accuracy 0.9941 and macro-F1 0.9902 (Table IV, Fig. 8). Salt-and-pepper noise is recognised perfectly, because isolated black and white pixels are unmistakable. The 218 errors (0.59%) fall into two groups (Table VIII): clean images taken for blurred or occluded ones, and low-severity occlusions taken for clean images. A 10% occlusion can hide in a dark image, and a clean photograph with large uniform dark areas resembles a masked one. Because the classes are imbalanced in the full protocol (clean has a third of the support of each

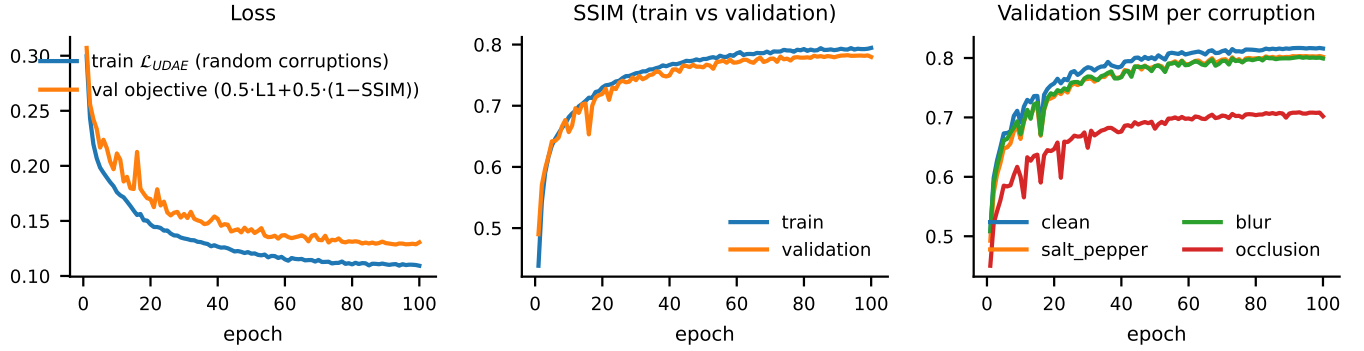


Fig. 4. Task 1 training and validation curves (100 epochs). Validation SSIM is reported separately for each corruption.

TABLE IV
Classifier metrics on the full test protocol.

Class	Support	Precision	Recall	F1
clean	3,669	0.965	0.977	0.971
salt-and-pepper	11,007	1.000	1.000	1.000
Gaussian blur	11,007	0.994	1.000	0.997
occlusion	11,007	0.998	0.988	0.993
macro average	36,690	0.989	0.991	0.990
overall accuracy			0.9941	

corruption), macro averages are reported next to the accuracy.

B. Specialist autoencoders

Design. Three specialists A_{salt} , A_{blur} , A_{occ} share the architecture of the Task 1 autoencoder (channels 48–384, bottleneck 32, 4.01 M parameters each) but have independently trained parameters, and each is trained only on its own corruption with the same clean target. Following the assignment’s allowance for feasibility, one shared Optuna search selects a common architecture: each trial trains all three specialists for six epochs on 1,000 images and scores the mean validation objective; learning rate, latent size, channels, batch size, dropout and α are tuned. The three specialists are then trained independently for 100 epochs on the full training set (learning rate 6.8×10^{-4} , batch 32, dropout 0.07, $\alpha = 0.60$).

Hard-routed inference. The restored image is

$$\hat{x} = \begin{cases} \tilde{x}, & r = \text{clean}, \\ A_{salt}(\tilde{x}), & r = \text{salt}, \\ A_{blur}(\tilde{x}), & r = \text{blur}, \\ A_{occ}(\tilde{x}), & r = \text{occlusion}, \end{cases} \quad (2)$$

so clean inputs take an identity bypass and are never processed by an expert. Two modes are evaluated: oracle routing uses the true label from the manifest (it measures the specialists alone), predicted routing uses the classifier (it measures the deployable system).

TABLE V
Colour preserved by each model (colourfulness of the output relative to the clean image) on 200 salt-and-pepper and 200 occlusion test inputs.

Model output	salt-and-pepper inputs	occlusion inputs
clean image (reference)	100%	100%
corrupted input	91%	77%
universal AE (Task 1)	97%	96%
specialist, first run ($\alpha \approx 0.3$)	43%	79%
specialist, final ($\alpha \approx 0.6$)	89%	88%
soft MoE, first run ($\alpha \approx 0.3$)	46%	94%
soft MoE, final ($\alpha \approx 0.6$)	97%	94%

C. Colour loss and its correction

Our first specialists (Optuna chose $\alpha = 0.29$, i.e. 71% of the loss on $1 - \text{SSIM}$) produced restorations that were visibly washed-out, sepia-toned versions of the input. Measuring colourfulness (mean of the per-pixel maximum minus minimum RGB channel) on 200 salt-and-pepper test inputs confirmed it: the first salt-and-pepper specialist kept only 43% of the clean image’s colour and the first soft mixture 46%, whereas the universal autoencoder kept 97% (Table V). SSIM compares local structure and is weakly sensitive to a global chroma shift, so a loss dominated by the SSIM term barely penalises discarding colour; the Optuna objective ($0.5 \mathcal{L}_1 + 0.5(1 - \text{SSIM})$) did not catch it either. We restricted the α search to $[0.55, 0.95]$ and repeated the specialist and mixture training: the final specialist ($\alpha = 0.60$) keeps 89% and the final mixture 97%. The effect is weaker on occlusion inputs (first specialist 79%, final specialist 88%, universal 96%, mixtures 94%), and the final occlusion specialist still keeps slightly less colour than the universal model. Table VI shows the side effect on the error measures: the L1 error of the deployed hard-routed system falls by about 18% (0.0456 to 0.0373) while SSIM is essentially unchanged, as expected when the loss again weights pixel accuracy. Both runs are kept in the repository (tags without and with _v2).

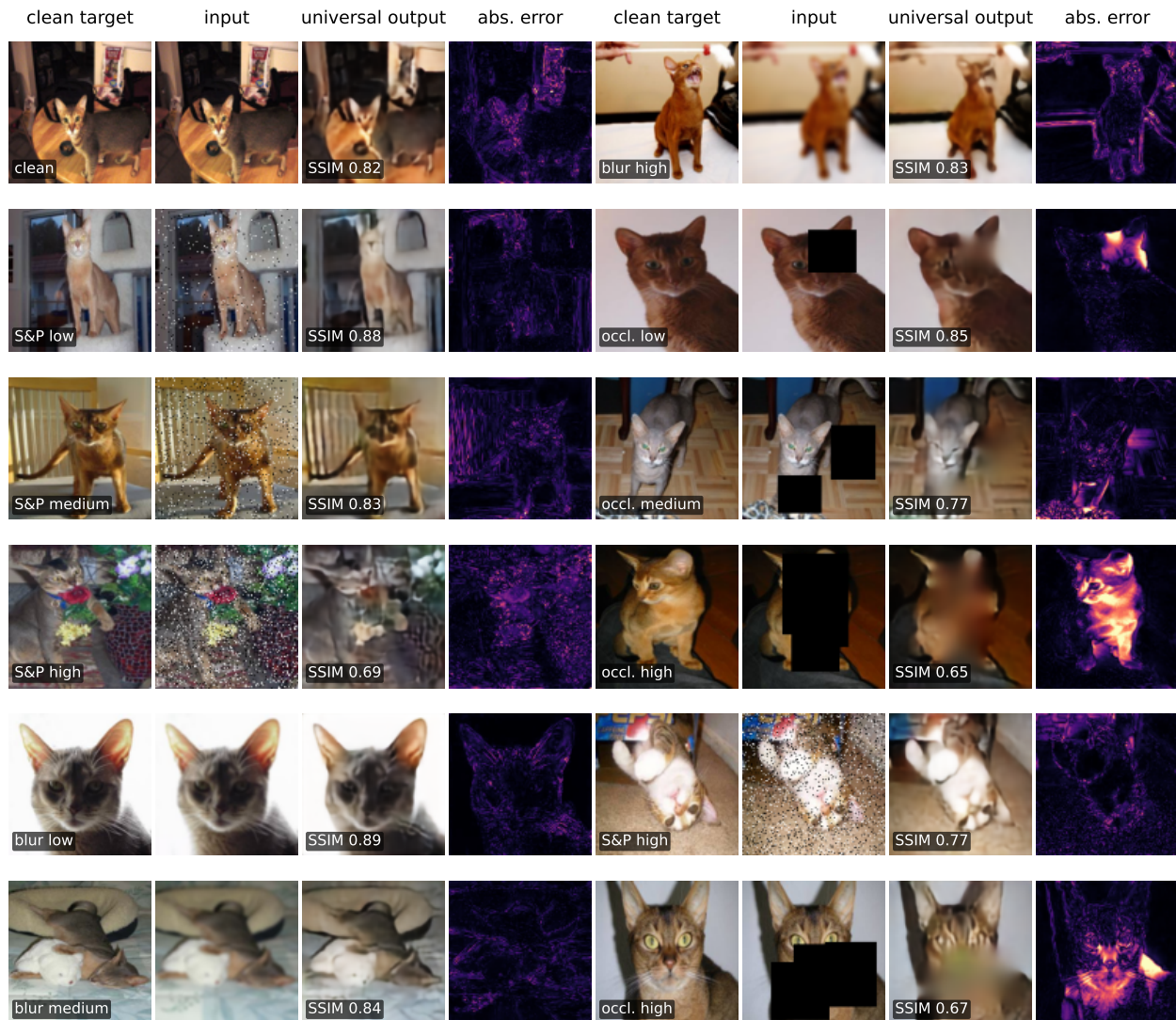


Fig. 5. Twelve representative test inputs for the universal autoencoder: clean target, corrupted input, output with its SSIM, and the absolute error map (every condition appears at least once).

TABLE VI
First run ($\alpha \approx 0.3$) versus final run ($\alpha \geq 0.55$) on the single-condition test manifest (3,669 inputs).

System	L1 (lower is better)		SSIM	
	first run	final	first run	final
Hard routing (oracle)	0.0454	0.0372	0.8170	0.8157
Hard routing (predicted)	0.0456	0.0373	0.8169	0.8157
Soft MoE	0.0456	0.0383	0.8221	0.8184

D. Routing results

Table VII compares the corrupted input, the universal autoencoder, oracle routing and predicted routing for each input type. Predicted routing (overall SSIM 0.779) is indistinguishable from oracle routing (0.779) because the classifier is accurate. Its advantage over the universal

model (0.772) is small and comes entirely from clean images (SSIM 0.996 against 0.814, which also raises the PSNR of the clean class far above the 26 dB of the universal model). On the corrupted types the specialists are slightly worse than the universal model in SSIM: 0.793 against 0.805 for noise, 0.780 against 0.790 for blur and 0.690 against 0.707 for occlusion (PSNR: 24.6, 24.5 and 21.1 dB against 25.8, 25.8 and 21.5 dB). A single specialist has the same capacity and the same bottleneck as the universal model, so specialisation brings no extra reconstruction ability; its hyper-parameters were chosen by one shared search for three models, and its batch size (32) gives half as many updates per epoch as the universal model's (16), which may explain part of the small gap. The measured benefit of routing is therefore the identity bypass.

TABLE VII

Hard routing (Task 2) on the full test protocol: SSIM and L1 of oracle versus predicted routing, and the share of inputs that the classifier mis-routes.

Input type	SSIM		L1		oracle	predicted	misroute %
	no restoration	universal	oracle	predicted			
clean	1.000	0.814	1.000	0.996	0.0000	0.0010	2.34
salt-and-pepper	0.382	0.805	0.793	0.793	0.0444	0.0444	0.00
Gaussian blur	0.814	0.790	0.780	0.780	0.0449	0.0449	0.01
occlusion	0.713	0.707	0.689	0.690	0.0597	0.0594	1.19
all (36,690)	0.672	0.772	0.779	0.779	0.0447	0.0447	0.59

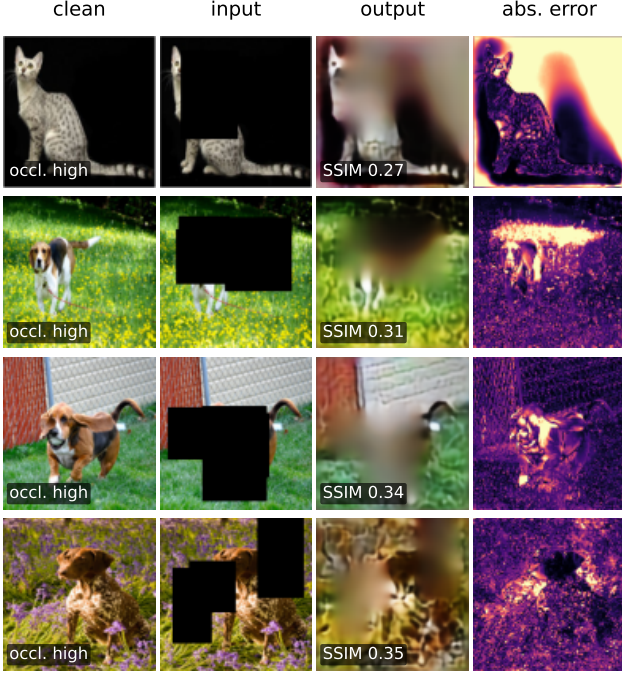


Fig. 6. The four lowest-SSIM corrupted test inputs for the universal autoencoder.

TABLE VIII

Classifier errors and their effect: change in SSIM of the predicted route relative to the oracle route for the mis-routed inputs.

True input	Routed to	Count	Mean Δ SSIM vs. oracle route
occlusion (low)	clean	126	+0.123
clean	Gaussian blur	68	-0.134
clean	occlusion	18	-0.252
occlusion (low)	Gaussian blur	3	-0.085
other (fewer than 3 each)	–	3	–
all mis-routed inputs	–	218	+0.009

Classifier-caused failures. The mis-routings have opposite consequences (Table VIII). The 86 clean images sent to a blur or occlusion expert are real failures: SSIM changes by -0.16 on average relative to the oracle route (-0.13 for those sent to the blur expert, -0.25 for those sent to the occlusion expert). In Fig. 9 a clean photograph of a dog on grass is taken for blurred and “restored” into a brown,

smeared scene (SSIM 0.51 instead of 1.00), a photograph with black letterbox bars and a cat on a black background are taken for occluded images and the occlusion expert invents content for the dark regions (0.54, 0.57), and a colourful image is taken for blurred (0.60). The 128 occlusions labelled “clean” (126 of them 10% masks) are left untouched, which scores about +0.12 SSIM higher than the oracle route, but only because SSIM ignores the untouched black rectangle (Section IV); the hole remains visible, so these are failures of the system even though the metric rewards them. Averaged over all inputs, mis-routing changes the mean SSIM by only +0.0001, because the two groups nearly cancel.

Other failures. Figure 11 shows the four lowest-SSIM outputs of the system: as for the universal model they are 35% occlusions that hide much of the animal (a beagle on grass, a basset hound, a dark terrier, a brown dog among flowers; SSIM 0.28–0.33). They are not routing errors. The specialist fills the hole with a smooth, brownish patch of texture even where the surroundings are green, a mild local colour error consistent with the lower colour retention of the occlusion specialist (Table V). Training curves of the classifier and of the specialists are in Fig. 12.

VI. Task 3: Jointly Trained Soft Mixture of Experts

A. Model and losses

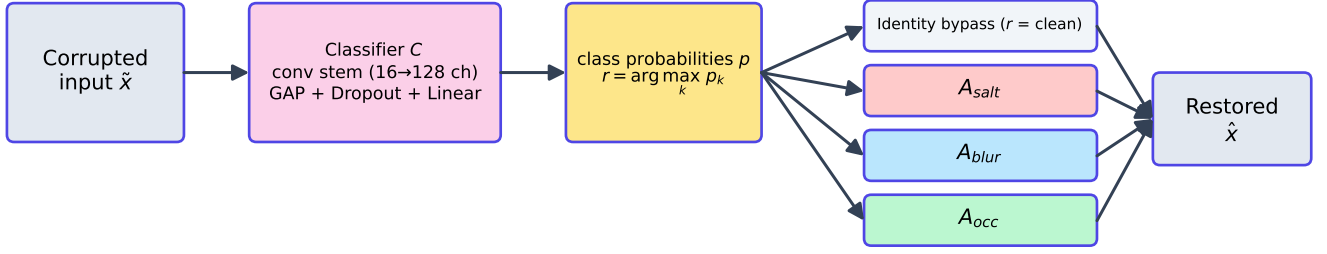
Hard routing fails when the classifier is uncertain or when two corruptions mix. The soft model (Fig. 7, bottom) replaces the decision by a differentiable combination. A gate G with the classifier’s architecture produces weights

$$w = \text{softmax}(G(\tilde{x})/\tau), \quad \hat{x} = w_0\tilde{x} + w_1A_{\text{salt}}(\tilde{x}) + w_2A_{\text{blur}}(\tilde{x}) + w_3A_{\text{occl}}(\tilde{x}) \quad (3)$$

where branch 0 is the identity and τ controls whether the weights are sharp or spread out. Because the mixture is differentiable, gate and experts are trained jointly from the reconstruction error. The components are not initialised randomly: the gate starts from the trained Task 2 classifier and the experts from the final specialists. Training has two stages: a short warm-up (2 epochs, learning rate 10^{-3}) in which the experts are frozen, in evaluation mode, and only the gate is trained, followed by 30 epochs of joint fine-tuning with a much smaller learning rate. The loss is

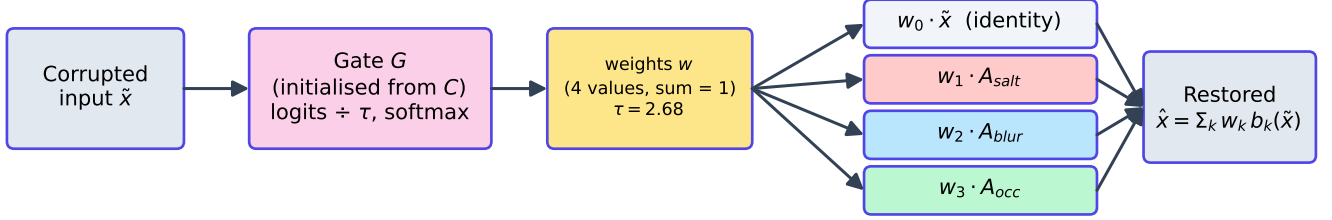
$$\mathcal{L}_{MoE} = \lambda_1\mathcal{L}_1 + \lambda_s(1 - \text{SSIM}) + \lambda_c\mathcal{L}_{CE} + \lambda_b\mathcal{L}_{\text{balance}}, \quad (4)$$

Task 2 - hard routing (classifier + specialists)



Specialists share one architecture (Task 1 autoencoder, 48→384 ch, bottleneck 32) but have independent weights

Task 3 - soft mixture of experts (jointly trained)



Loss: $\lambda_1 \mathcal{L}_{L1} + \lambda_s(1 - \text{SSIM}) + \lambda_c \mathcal{L}_{CE} + \lambda_b \mathcal{L}_{balance}$; stage 1: experts frozen, gate only; stage 2: all unfrozen, smaller lr

Fig. 7. Top: hard routing (Task 2). A classifier chooses the corruption class r , which selects a specialist or the identity bypass. Bottom: the soft mixture of experts (Task 3), built from the same components.

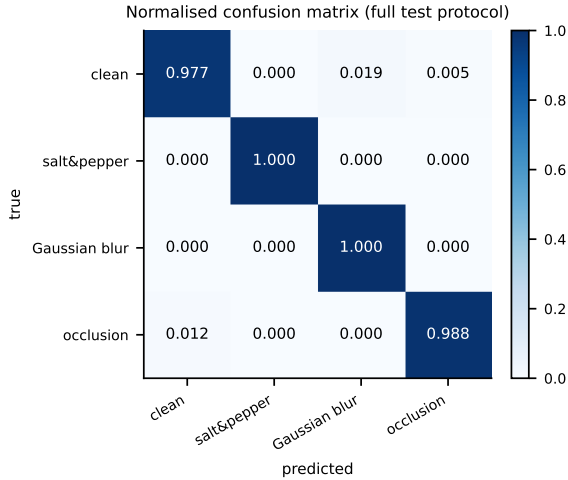


Fig. 8. Row-normalised four-class confusion matrix of the corruption classifier.

with $\mathcal{L}_{balance} = \sum_{k=0}^3 (\bar{w}_k - \frac{1}{4})^2$, where $\bar{w}_k$ is the mean weight of branch k in a mini-batch. The cross-entropy acts on the raw gate logits, which keeps the question “does the gate know the true class” separate from “how sharply does it mix” (the temperature). We tie $\lambda_1 = \alpha$ and $\lambda_s = 1 - \alpha$, so that the five quantities Optuna tuned (joint learning rate, temperature, λ_c , λ_b and the L1/SSIM

weighting α) map one-to-one onto the assignment’s list and the reconstruction weights cannot both shrink to zero. The selected values are $\tau = 2.68$, $\lambda_c = 0.48$, $\lambda_b = 1.20$, $\alpha = 0.61$ and a joint learning rate of 4.3×10^{-5} ; median pruning removed half of the trials (Section VIII). The first mixture (with $\alpha = 0.32$) inherited the colour loss of its experts, and the final one was retrained from the corrected specialists (Section V-C).

B. Reconstruction results

Table IX compares the three restoration systems on every test condition. The soft mixture reaches an overall SSIM of 0.783 against 0.779 for predicted hard routing and 0.772 for the universal model. Clean images stay almost untouched (SSIM 0.991) because the gate puts most of its weight on the identity branch. On the corrupted types the mixture is on par with hard routing and with the universal model (overall 0.793 for noise, 0.790 for blur and 0.697 for occlusion, PSNR 24.8, 25.0 and 21.1 dB); it is slightly better than hard routing at the low severities (for example blur $k = 3$: 0.820 against 0.792) and slightly worse at the high ones (noise $p = 0.15$: 0.778 against 0.790), which is what a smoother, less committed decision would be expected to do. Blending experts does not remove the fundamental limit on heavy occlusion.

TABLE IX
Soft mixture of experts (Task 3) compared with the universal model and predicted hard routing on the full test protocol.

Corruption	Level	SSIM			soft MoE	
		universal	hard (pred.)	soft MoE	L1	PSNR (dB)
clean	—	0.814	0.996	0.991 ± 0.017	0.0081	40.19
salt-and-pepper	low	0.813	0.796	0.807 ± 0.070	0.0427	24.98
salt-and-pepper	medium	0.808	0.794	0.795 ± 0.075	0.0436	24.83
salt-and-pepper	high	0.796	0.790	0.778 ± 0.075	0.0441	24.71
Gaussian blur	low	0.814	0.792	0.820 ± 0.074	0.0400	25.55
Gaussian blur	medium	0.805	0.790	0.795 ± 0.086	0.0424	25.04
Gaussian blur	high	0.751	0.757	0.754 ± 0.096	0.0448	24.49
occlusion	low	0.768	0.747	0.771 ± 0.083	0.0484	22.78
occlusion	medium	0.718	0.699	0.699 ± 0.093	0.0575	21.28
occlusion	high	0.634	0.625	0.621 ± 0.098	0.0718	19.16
all (36,690)		0.772	0.779	0.783 ± 0.120	0.0443	25.30

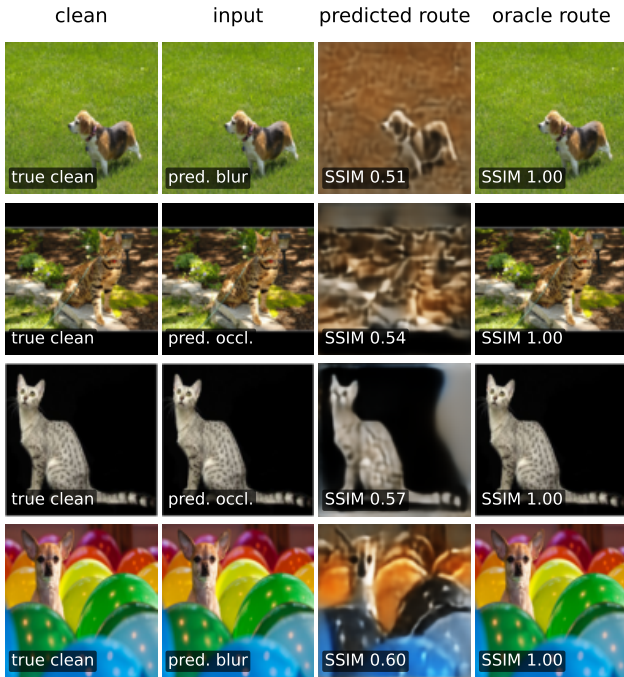


Fig. 9. Classifier-induced failures: the four mis-routed inputs with the largest SSIM loss, showing the output of the predicted route next to the oracle route (all four are clean images).

C. Behaviour of the gating network

Average weights. Table X and Fig. 14 give the mean routing weight of each branch for every true corruption and severity. The gate is well aligned with the truth: the matching expert receives on average 0.86 of the weight for salt-and-pepper, 0.85 for blur and 0.92 for occlusion, and the identity branch 0.81 for clean images. The weight grows with severity (occlusion: from 0.80 at 10% coverage to 0.99 at 35%), as it should, since heavier corruption is easier to recognise. Weights are not one-hot because the temperature (2.68) deliberately spreads them: the mean

TABLE X
Mean gate weights by true condition (full test protocol).

True condition	w_0 identity	w_1 salt	w_2 blur	w_3 occl.	dominant
clean	0.808	0.030	0.098	0.063	clean
salt-and-pepper low	0.052	0.838	0.076	0.034	salt-and-pepper
salt-and-pepper medium	0.041	0.865	0.068	0.026	salt-and-pepper
salt-and-pepper high	0.042	0.864	0.067	0.027	salt-and-pepper
Gaussian blur low	0.148	0.017	0.807	0.028	Gaussian blur
Gaussian blur medium	0.096	0.014	0.865	0.025	Gaussian blur
Gaussian blur high	0.096	0.014	0.864	0.025	Gaussian blur
occlusion low	0.108	0.064	0.028	0.800	occlusion
occlusion medium	0.012	0.026	0.005	0.958	occlusion
occlusion high	0.002	0.009	0.001	0.988	occlusion

largest weight is 0.868.

Dominant versus distributed routing. Figure 15 contrasts inputs on which one expert dominates (left: the most “one-expert” input of each corruption) with inputs on which the weights are spread (right). The spread-out cases are low-severity corruptions, for which the image is close to a clean one and the identity and expert branches are both plausible: the mean weight entropy is 0.58 nats for 10% occlusions and 0.64 for clean images, against only 0.06 for 35% occlusions, where the gate is nearly one-hot (Table X). For such inputs the mixture yields a smoother output than a hard decision would.

Inactive or dominant experts. Table XI lists, for each true type, the percentage of inputs on which each branch has the largest weight. No branch is dead: each branch is the top branch for a substantial share of inputs (overall: identity 10.5%, salt-and-pepper 30.0%, blur 30.1%, occlusion 29.4%), and no branch dominates unrelated inputs, so the balance regulariser did its job without forcing a uniform mixture.

Failure cases. Figure 17 shows the four worst outputs of the mixture (SSIM 0.24–0.31). They are the same kind of input as for the other systems: 35% occlusions that hide the subject. The first is the cat on a black background whose black mask merges with the background, so the input looks like a half-empty image and the mixture produces a grey-brown cloud; in the other three (a beagle, a basset hound and a dark terrier on grass) the gate gives

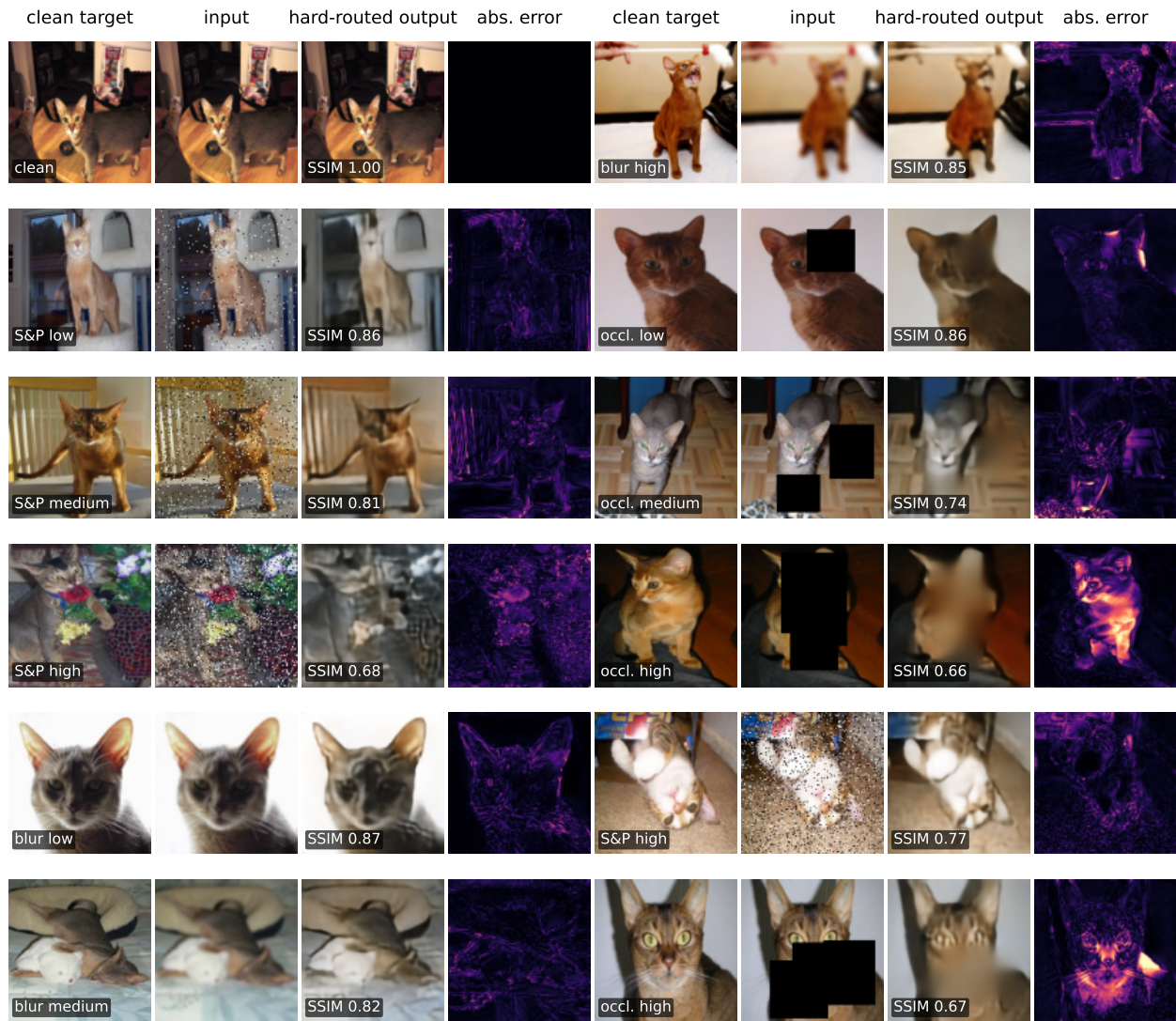


Fig. 10. Twelve representative test inputs for hard-routed restoration (predicted routing).

TABLE XI

Dominance analysis: where each branch has the largest weight.

True type	% of inputs where branch has the largest weight				mean max weight
	identity	salt	blur	occl.	
clean	99.1	0.0	0.7	0.2	0.811
salt-and-pepper	0.0	100.0	0.0	0.0	0.856
Gaussian blur	0.0	0.0	100.0	0.0	0.846
occlusion	1.9	0.0	0.0	98.0	0.921
all inputs	10.5	30.0	30.1	29.4	0.868

the occlusion expert nearly all of the weight, which is the correct routing, but the expert cannot recreate the animal. The worst cases of the mixture are about 0.03–0.04 SSIM below those of the universal model (0.24–0.31 against 0.27–0.35). Because the information is absent, no routing or blending strategy can recover these inputs, and all three systems produce a smooth, plausible fill instead

of the subject.

VII. Task 4: Style-Conditioned Face-to-Sketch cGAN

A. Model

The generator $\hat{y} = G(x, s)$ (Fig. 18) is a U-Net [15]: seven stride-2 encoder blocks (64, 128, 256, 512 channels, four times 512 at the deepest levels) reduce the 128×128 photograph to a 1×1 code, and six transposed-convolution blocks with skip connections restore the resolution; a final transposed convolution with tanh produces a one-channel sketch. The style $s \in \{1, 2, 3\}$ is a learned categorical embedding (three vectors of 32 values). It is incorporated into the generator twice, broadcast over the image as extra input channels and concatenated to the 1×1 bottleneck, and into the discriminator as extra input channels; it is therefore part of the model and not merely an interface label. The

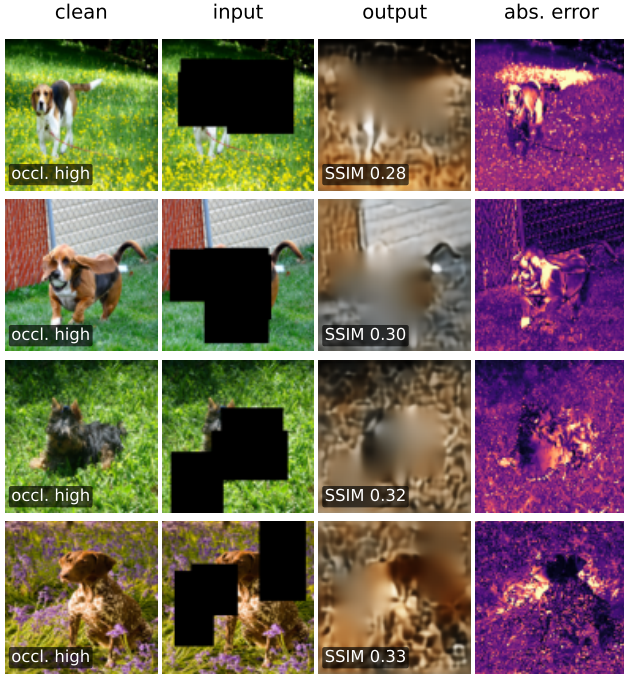


Fig. 11. The four lowest-SSIM corrupted test inputs under predicted routing.

generator has 42.1M parameters. The discriminator is a PatchGAN [14], [16]: it receives the photograph, the style map and either the real or the generated sketch (36 channels), and outputs a 14×14 map of patch logits (2.8M parameters).

B. Losses and training

The discriminator is trained with binary cross-entropy on logits to label real pairs (x, y, s) as one and generated pairs $(x, G(x, s), s)$ as zero; its real and fake losses are recorded separately. The generator minimises

$$\mathcal{L}_G = \mathcal{L}_{adv} + \lambda_{L1} \mathcal{L}_{L1}(y, G(x, s)), \quad (5)$$

the adversarial term being the discriminator’s error on generated pairs. Both networks use Adam ($\beta_1 = 0.5$, $\beta_2 = 0.999$). Optuna tuned the generator and discriminator learning rates, batch size, base channel count, dropout, style-embedding size and λ_{L1} with 15-epoch trials on all 899 training pairs (three completed, five pruned). The selected configuration (generator learning rate 1.5×10^{-4} , discriminator learning rate 8.6×10^{-4} , batch 8, 64 base channels, dropout 0.30, embedding size 32, $\lambda_{L1} = 121$) was then trained for the full 100 epochs, recording the discriminator real and fake losses, the generator adversarial and L1 losses and the validation L1, SSIM and PSNR separately for every epoch (Fig. 19). Every ten epochs the same eight validation photographs are rendered in all three styles and logged to MLflow (Fig. 20), so the development of the generator can be inspected over time.

TABLE XII

Task 4 on the official FS2K test set. The last row is a second training recipe (photo-only brightness/contrast jitter and learning-rate decay) that was tried and rejected because it was worse.

Test subset	Pairs	L1	SSIM	PSNR (dB)
Style 1	619	0.0791	0.534 ± 0.107	17.55
Style 2	381	0.1522	0.395 ± 0.082	12.47
Style 3	46	0.0656	0.600 ± 0.074	18.90
all (selected model)	1046	0.1051	0.486	15.76
all (rejected variant)	1046	0.1112	0.451	15.23

C. Results

On the 1,046 official test pairs the selected model reaches L1 0.1051, SSIM 0.486 and PSNR 15.76 dB (Table XII, Fig. 21). The error differs strongly between styles: style 2 has an L1 error of 0.152 against 0.079 for style 1 and 0.066 for style 3, because its reference sketches use heavy dark strokes which a soft generator cannot match pixel by pixel, while style 1 and style 3 are lighter. The test styles are imbalanced (619, 381 and 46 pairs), so the style-3 figures rest on few samples. Each style is visibly different for the same photograph (Fig. 22): style 2 is darkest and heaviest, style 1 lightest.

Why the sketches are soft. Compared with the real sketches the generated ones are smoother (Fig. 21); we measured it. The average gradient magnitude of the generated sketches is only 37% of that of the real sketches (Table XIII). A sharper input helps a little and a blurred input hurts a little (Table XIV), but the effect is small next to the 63% gap, so the limit lies in the training rather than in the 128×128 input. Two causes are consistent with the evidence. With only 899 training pairs and $\lambda_{L1} = 121$ the L1 term dominates and regresses towards an average stroke; and in the selected Optuna trial the discriminator stayed close to chance for the whole 15 epochs (real and fake loss near $\ln 2 \approx 0.69$), so the adversarial term contributed little; in the 100-epoch run it did learn to separate real from fake (losses fell to about 0.4 and the generator’s adversarial loss rose to about 2, Fig. 19), but the generator’s L1 term still dominates the objective. This is the perception–distortion trade-off [8]: pixel metrics reward smooth outputs. A second recipe (photo-only brightness and contrast jitter, linear learning-rate decay) did not help (SSIM 0.451, L1 0.1112) and was discarded. Stronger adversarial or perceptual losses, more data, or a higher resolution are the obvious next steps and were not explored.

Failure cases. Figure 23 shows the four test pairs with the lowest SSIM (0.18–0.21). Three are style 2 portraits with long hair: the reference sketches draw the hair with dense, dark, hand-drawn strokes, while the generator renders it as a smooth grey mass and draws the face softly (SSIM 0.18, 0.20, 0.20). The fourth is a style 1 portrait with long hair and bangs, where the reference consists

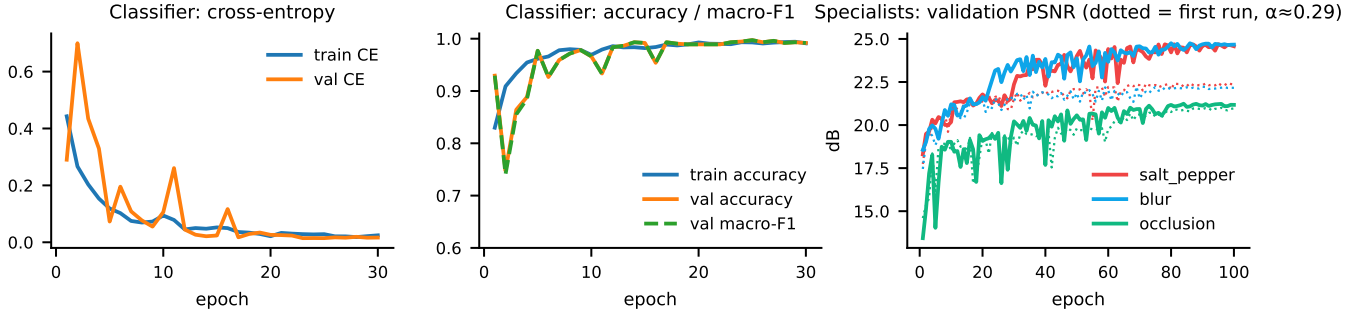


Fig. 12. Task 2 training curves: classifier loss and accuracy, and validation PSNR of the three specialists in the first run (dotted) and the final run (solid).

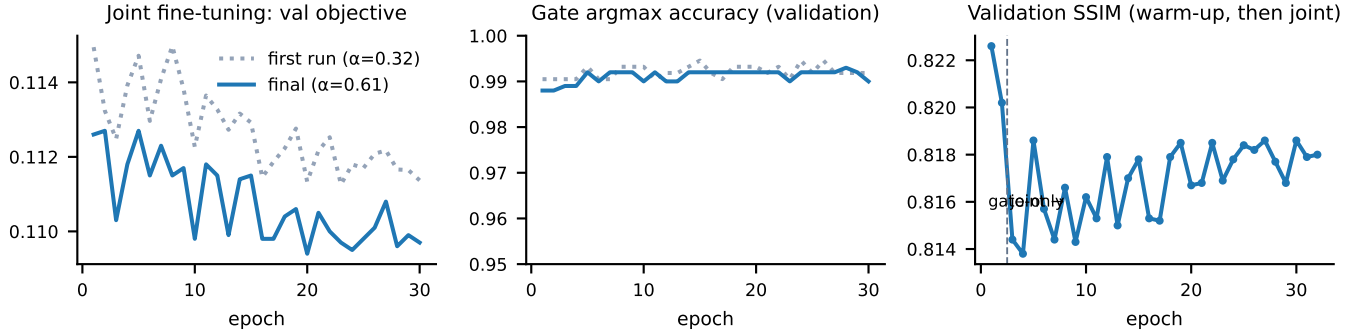


Fig. 13. Task 3 training: validation objective and gate accuracy for the first (dotted) and final runs, and validation SSIM over the warm-up and joint stages of the final run.

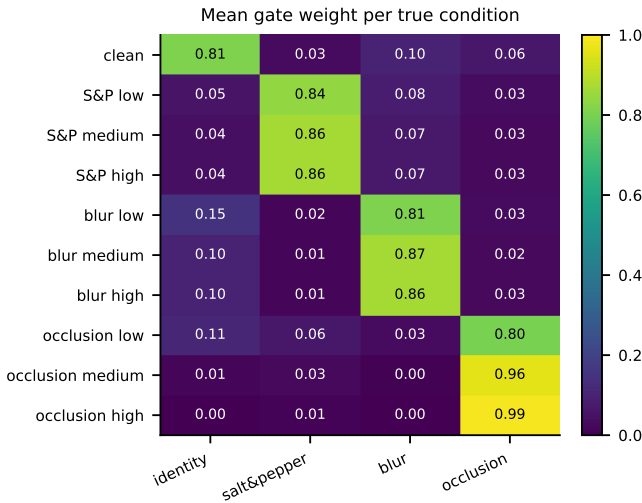


Fig. 14. Routing heat-map: mean weight of each branch for each true corruption and severity.

of thin, crisp lines and the generated sketch is pale and blurred (0.21). The faces remain recognisable, so these are failures of stroke detail, not of identity. The error maps light up along every reference stroke: a thin line that is missing or displaced by a few pixels costs a large pixel

TABLE XIII
Edge strength (mean gradient magnitude) of real and generated sketches on the FS2K test set.

Test subset	Pairs	Real sketch edge strength	Generated	Generated / real
Style 1	619	0.0735	0.0267	36%
Style 2	381	0.1506	0.0554	37%
Style 3	46	0.0613	0.0368	60%
all	1046	0.1011	0.0376	37%

TABLE XIV
Sensitivity of the generated edge strength to the sharpness of the input photograph.

Input photo (300 test faces)	Generated edge strength	L1 to real sketch
input blurred (Gaussian, $r=1.5$)	0.0346	0.1108
input as is	0.0381	0.1084
input sharpened (unsharp mask)	0.0436	0.1091

error, which is why pixel metrics punish a soft generator so heavily on hair-rich, style 2 subjects.

VIII. Hyper-parameter Optimisation with Optuna

Every task has its own Optuna study (TPE sampler, seed 42, studies stored in SQLite under `optuna_studies/`, every trial also logged to MLflow). Trials are deliberately short and use sub-samples of the training data so that the search is affordable; the selected configuration is

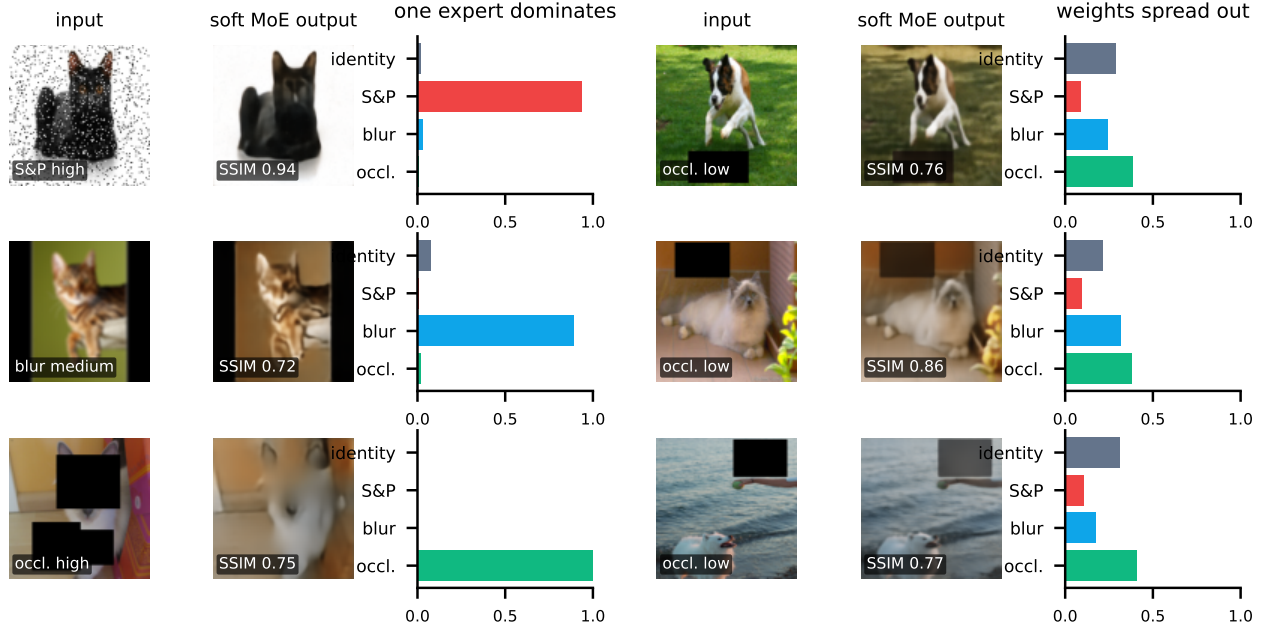


Fig. 15. Left: inputs for which a single expert dominates the mixture. Right: inputs for which the weights are distributed over several branches.

then retrained on all training data for the full schedule. Table XV lists the complete search spaces, Table XVI the number of completed and pruned trials and the best trial of each study, Table XVII the final configurations, and Fig. 24 the study histories. Pruning (median rule) stopped poorly performing trials early in Tasks 1, 2 (classifier), 3 and 4; the shared specialist search is not pruned because each trial trains three models and offers no single intermediate value.

Three search-related lessons are worth recording. First, short trials favour configurations that learn fast early, which is how the first Task 1 search selected a high dropout rate that later under-fitted (Section IV). Second, an objective made of L1 and SSIM cannot see colour loss, so the first specialist search selected an SSIM-heavy α that later discarded colour (Section V-C); constraining a parameter range after inspecting outputs is legitimate only when it is reported, as it is here. Third, with 8–12 trials per study the searches are compact: they find good, not optimal, settings.

IX. Application Design, Deployment and Tracking

A. Interface design in Google Stitch

The interface was designed first in Google Stitch [24]. The first prompt described four workspaces (one per task) with a sidebar, input and corruption cards, result panels and a system-information area; Stitch returned four screens and a design system (indigo primary colour, Inter for text and JetBrains Mono for numbers, white rounded cards on a light background). Using the first screens, a usability problem became clear: the controls

and the Run button were below the fold, so the user scrolled down to choose options and back up to see the result. A second prompt asked for a layout in which everything needed for one run fits on a laptop screen: a narrow control panel on the left with the Run button pinned at its bottom, large results on the right, and extra options collapsed. Both rounds are shown in Fig. 25 and kept in the repository (stitch_design/, with the exported HTML and DESIGN.md). The exported HTML is a static mock-up with invented data (a GPU model, LPIPS values, stroke-intensity sliders) that our back end cannot provide, so we took its structure, colours and typography and implemented it as React components that show only real data. One exported screen of the second round (sketch workspace) was empty, which is stated in the figure. A screenshot of the Stitch project page itself, with the generated screens of both rounds on the canvas and the prompt history on the left, is shown in Fig. 26; the wording of the prompts is kept in stitch_design/PROMPTS.md.

B. Front end

The front end is a React 18 single-page application built with Vite and styled with Tailwind CSS. The four workspaces (Universal Restoration, Hard-Routed Restoration, Soft Mixture-of-Experts Restoration, Face-to-Sketch Generator) share one layout (Fig. 27). The user supplies an image by upload (or drag and drop), by choosing one of ten built-in test samples, or — for sketches — by webcam capture; chooses a corruption type and severity (low, medium, high or custom sliders and a random seed, so the same seed reproduces the same corruption) or leaves

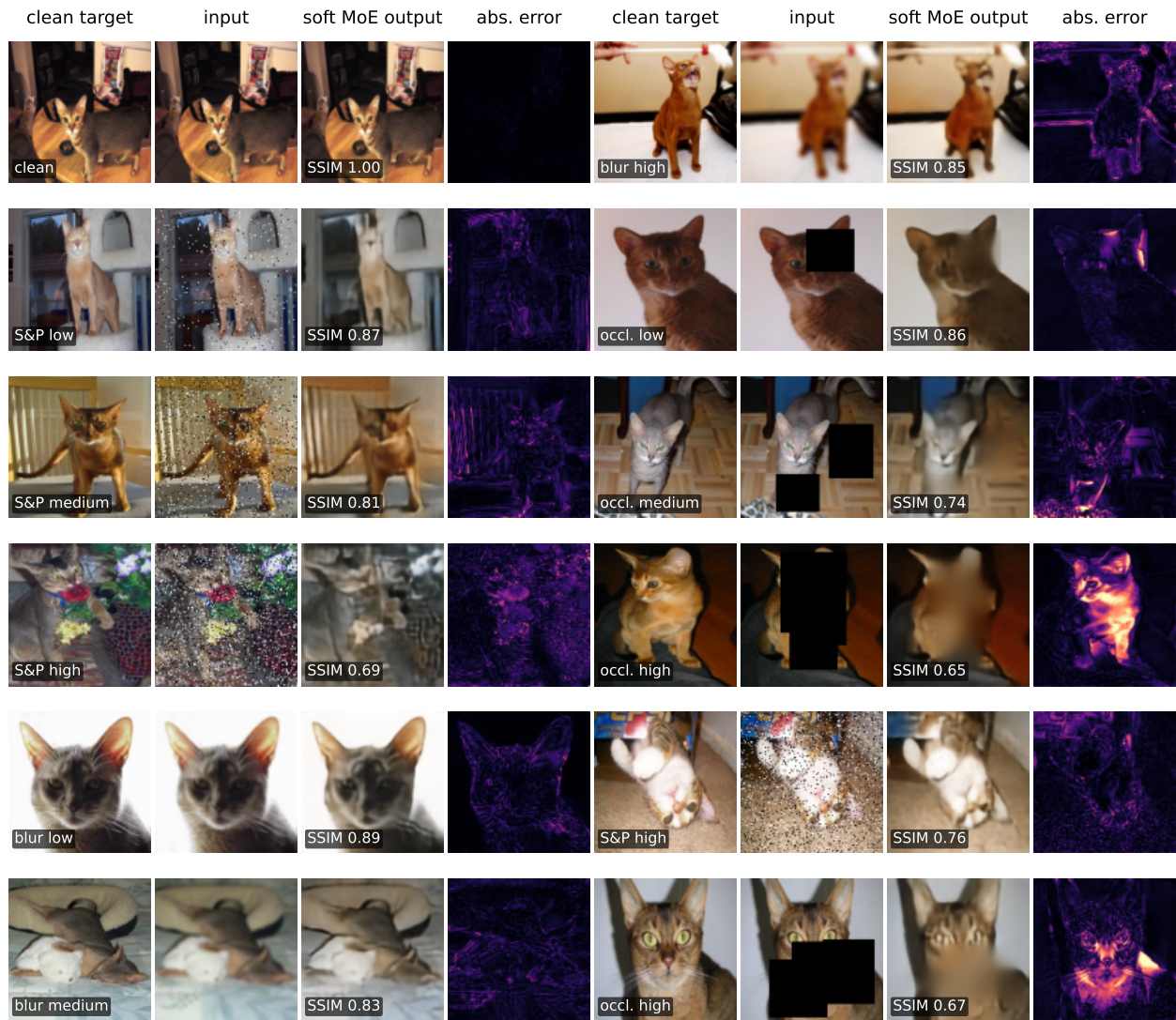


Fig. 16. Twelve representative test inputs for the soft mixture of experts.

the image as is to restore an already corrupted upload; and presses Run. The result panels show the model input, the restored output and the absolute error map (when a clean reference exists), each with a PNG download, together with the information the assignment asks for: the inference time; for hard routing, the four classifier probabilities, the predicted corruption, the selected expert, a confidence value and the routing mode (predicted or oracle); for the mixture, the four routing weights, a stacked contribution bar, the dominant branch and the weight entropy; and for sketches, the style, side-by-side photograph and sketch with a download button. A measurement with a real browser showed that at 1366×768 and 1920×1080 pixels the Run button is always visible and the page never scrolls.

C. Back end

The FastAPI [22] service loads all seven ONNX models once at start-up and exposes: GET /api/health (model status, runtime and device), GET /api/samples, POST /api/restore/universal, /restore/hard, /restore/soft and POST /api/sketch. Every image endpoint validates the upload (allowed content types, at most 10 MB and 40 megapixels, decodable image, path-traversal-safe sample names, style in {1, 2, 3}, allowed corruption and slider ranges), preprocesses it exactly as in training (bicubic resize to 128×128 ; for sketches an optional centre crop, padding or stretch), applies the requested corruption with a NumPy port of the training corruption code (checked against it by an automated test), runs the model(s) with ONNX Runtime on the CPU, and returns PNG data URLs together with the routing information, timings and, when a clean reference exists, PSNR before and after and an

TABLE XV

Complete Optuna search spaces (log = log-uniform, U = uniform, { } = categorical). Channel options c16, c32, c48 denote the four-stage widths (16, 32, 64, 128), (32, 64, 128, 256) and (48, 96, 192, 384).

Study	Searched parameters	Trial budget and objective
Task 1 (final)	lr log[3e-4, 3e-3]; batch {16, 32, 64}; latent channels {32, 64, 128, 256}; channels {c16, c32, c48}; dropout U[0, 0.15]; α U[0.2, 0.95]	12 trials $\times$ 6 epochs on 1,000 images; validation $0.5\mathcal{L}_1 + 0.5(1 - \text{SSIM})$
Task 1 (first)	as above but dropout U[0, 0.3] and latent channels {16, 32, 64, 128}	8 trials $\times$ 3 epochs on 600 images
Task 2 classifier	lr log[3e-4, 3e-3]; batch {16, 32, 64}; channels {c16, c32, c48}; dropout U[0, 0.5]; weight decay log[10 ⁻⁶ , 10 ⁻²]	8 trials $\times$ 4 epochs on 1,000 images; validation macro-F1 (maximised)
Task 2 specialists	as Task 1; α U[0.2, 0.95] (first run) and U[0.55, 0.95] (final run)	8 trials, each training all three specialists for 6 epochs on 1,000 images; mean validation objective
Task 3	joint lr log[10 ⁻⁵ , 5e-4]; temperature τ log[0.3, 3]; λ_c log[0.01, 2]; λ_b U[0, 2]; α U[0.2, 0.95] (first) and U[0.55, 0.95] (final)	8 trials $\times$ (1 warm-up + 3 joint epochs) on 1,000 images; validation objective
Task 4	lr _G , lr _D log[5e-5, 10 ⁻³]; batch {8, 16, 32}; base channels {32, 48, 64}; dropout U[0, 0.5]; embedding size {8, 16, 32}; λ_{L1} log[10, 200]	8 trials $\times$ 15 epochs on all 899 pairs; validation $0.5 L1 + 0.5(1 - \text{SSIM})$

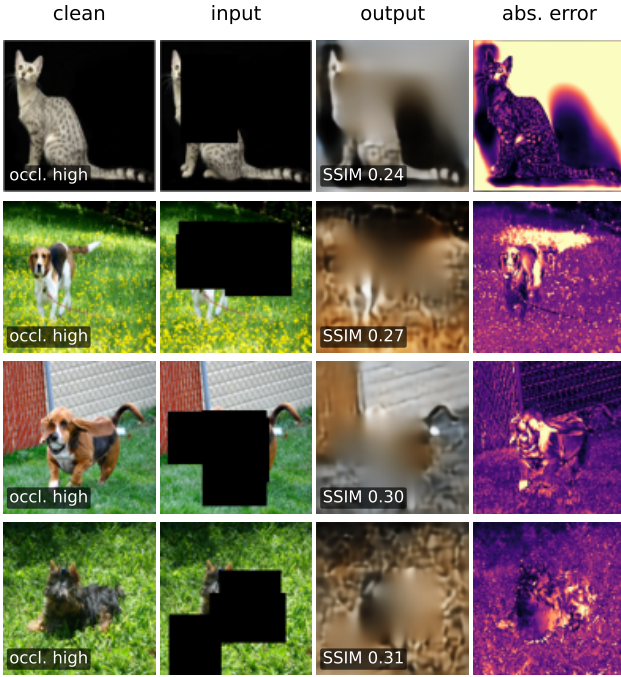


Fig. 17. The four lowest-SSIM corrupted test inputs for the soft mixture.

error map. A missing model file produces a clear 503 message and a warning in the interface instead of a crash.

D. ONNX export and verification

All models needed for inference were exported to ONNX (opset 17, dynamic batch) by `src/export/*`; each export is checked with the ONNX checker and compared with the

TABLE XVI

Optuna studies: trials, completed and pruned counts, best trial (studies marked “first” or “rejected” are kept as evidence).

Study	Trials	Completed	Pruned	Best trial	Best value
Task 1 universal AE	12	6	6	#9	0.2622
Task 2 classifier	8	4	4	#1	0.9243
Task 2 specialists (first run)	8	8	0	#6	0.2826
Task 2 specialists (final, $\alpha \geq 0.55$)	8	8	0	#6	0.2816
Task 3 soft MoE (first run)	8	4	4	#0	0.1135
Task 3 soft MoE (final, $\alpha \geq 0.55$)	8	4	4	#0	0.1108
Task 4 cGAN (selected)	8	3	5	#0	0.3139
Task 4 cGAN (rejected variant)	12	5	7	#0	0.3077

TABLE XVII

Selected configuration of each model.

Model	Selected configuration
Task 1	lr=4.61e-04, batch_size=16, bottleneck_dim=32, channels=c48, dropout=1.04e-03, alpha=0.583
Classifier	lr=1.53e-03, batch_size=32, channels=c16, dropout=0.152, weight_decay=1.26e-04
Specialists	lr=6.85e-04, batch_size=32, bottleneck_dim=32, channels=c48, dropout=0.071, alpha=0.598
Soft MoE	lr=4.33e-05, temperature=2.678, w_cls=0.483, w_bal=1.197, alpha=0.612
cGAN	lr_g=1.54e-04, lr_d=8.63e-04, batch_size=8, base=64, dropout=0.301, emb_dim=32, lambda_l1=121.069

PyTorch output on a random batch, and must agree within an absolute tolerance of 10^{-4} (10^{-3} for the generator); the observed differences are all below 2.1×10^{-6} (Table XVIII). The whole soft mixture (gate, three experts, identity and mixing) is one ONNX graph that returns both the image and the four weights; only the generator of the cGAN is exported. Beyond the per-model check, we re-measured

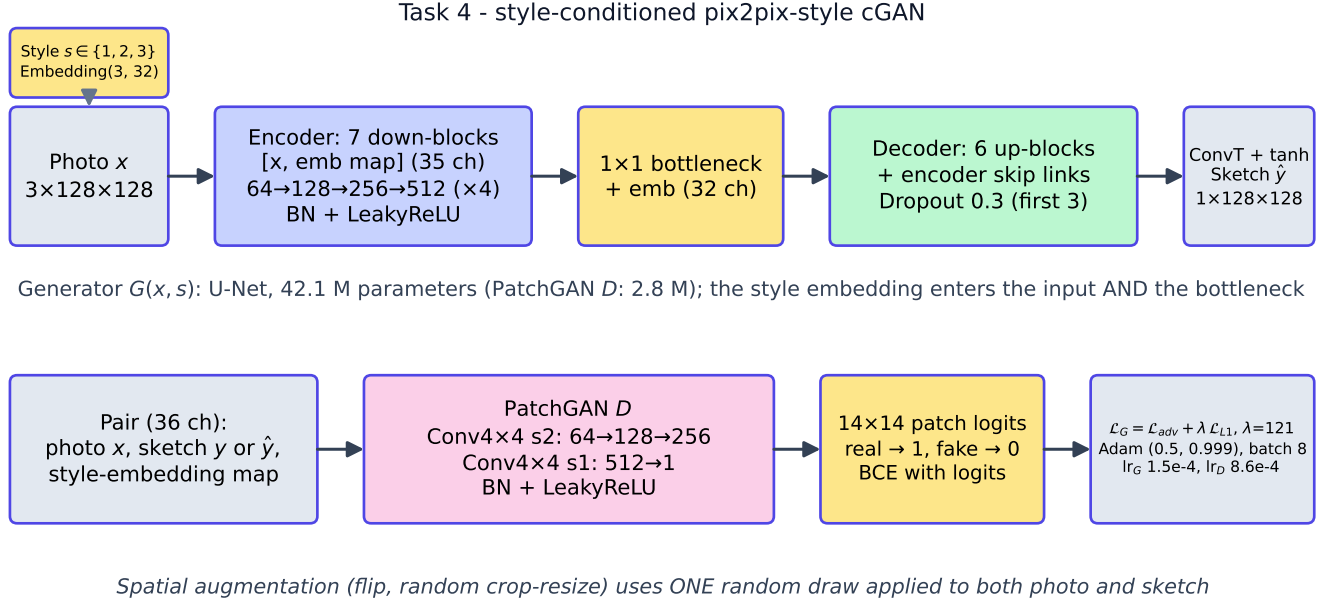


Fig. 18. Style-conditioned cGAN: U-Net generator with a style embedding in the input and the bottleneck, and PatchGAN discriminator.

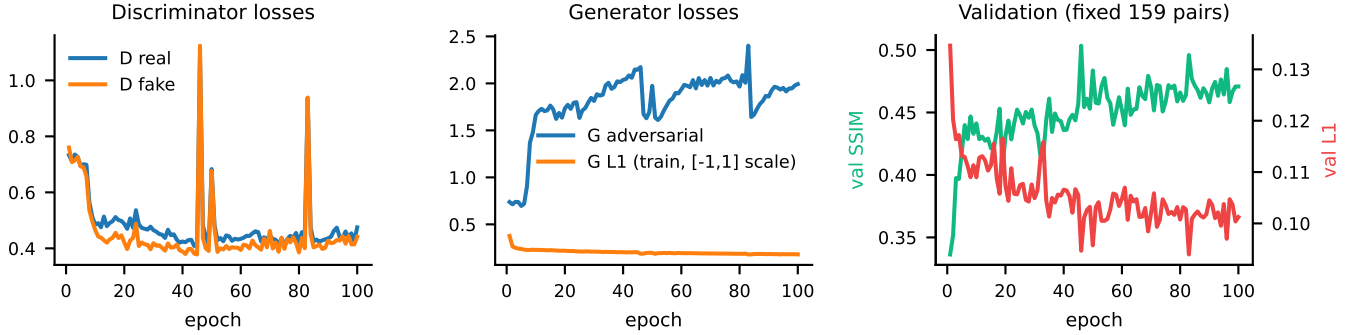


Fig. 19. Task 4 training: discriminator and generator losses, and validation SSIM and L1 (fixed validation pairs).

TABLE XVIII

ONNX export and verification: file size and maximum absolute difference between PyTorch and ONNX Runtime outputs on a random batch (checked by the export scripts); typical single-image CPU latency.

Model	Size (MB)	Max PyTorch-ONNX	Latency (ms)
Task 1 universal autoencoder	16.1	5.4e-7	12.4
Task 2 classifier	1.2	2.0e-6	0.3
Task 2 specialist (salt-and-pepper)	16.1	5.7e-7	15.3
Task 2 specialist (blur)	16.1	4.5e-7	12.4
Task 2 specialist (occlusion)	16.1	5.4e-7	15.3
Task 3 soft mixture (whole pipeline)	49.4	2.4e-7	39.8
Task 4 generator	168.5	1.4e-6	21.1

the published test results using only the deployed artefacts: the ONNX files, the back end’s NumPy corruptions and a different SSIM implementation (scikit-image). The SSIM values of the universal and mixture models agreed

with the PyTorch evaluation to four decimals on the first 300 test entries, and the classifier accuracy on the full first manifest was identical (0.9910).

E. Containerised deployment

Two containers are built from the repository: the back end (Python, FastAPI, ONNX Runtime) and the front end (an nginx image serving the built application and proxying /api to the back end). The model files are mounted read-only from `./onnx_models`; they are not stored in Git, and `scripts/download_models.py` fetches them from the release assets of the repository. One command,

`docker compose up --build`

starts the application at `http://localhost:3000`; the back end has a health check, and an optional profile starts the

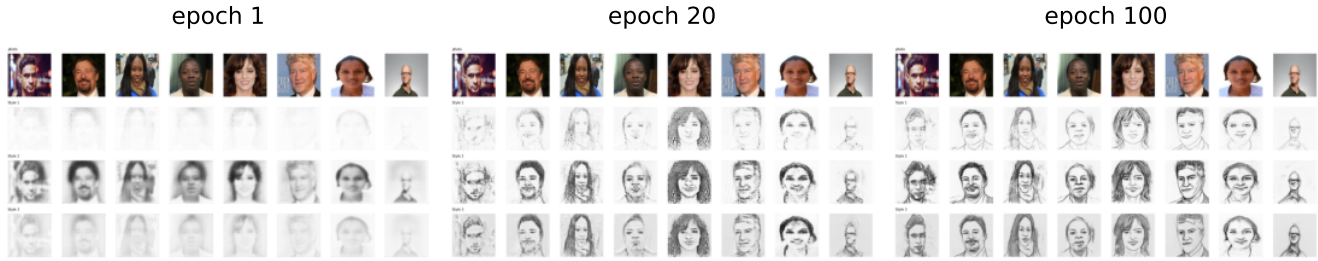


Fig. 20. The same eight validation photographs (top row of each panel) rendered by the generator in styles 1, 2 and 3 after epochs 1, 20 and 100.

MLflow interface on port 5000. An end-to-end smoke test (scripts/smoke_test_stack.py) exercises every endpoint, corruption, severity, routing mode, framing option and error path through the nginx address, as a browser would (54 checks), and the back end has its own 38 automated tests.

F. Experiment tracking

Training runs, Optuna trials, hyper-parameters, per-epoch losses and metrics, the best checkpoints and visual outputs (for Task 4 the fixed-photo sample grids every ten epochs) are recorded with MLflow [20] in five experiments: the universal autoencoder, the classifier, the specialists, the mixture and the cGAN. The MLflow database and the Optuna SQLite studies are part of the repository, so the records can be opened without retraining (Fig. 28). Checkpoint files are not stored in Git because of their size. [Optionally add your own MLflow screenshots if you prefer them to the automatically captured ones.]

X. Limitations and Deviations

We list what a reader should not over-interpret and where our procedure deviates from an ideal one.

- The universal model damages inputs that need no restoration. The bottleneck cannot reproduce fine detail, so clean and mildly blurred images score below the untouched input (Table III); the routing systems exist to avoid this.
- Specialisation did not improve the corrupted cases. With equal capacity and bottleneck, the specialists are slightly worse than the universal autoencoder on their own corruption (by 0.01–0.02 SSIM, Table VII); the measured benefit of routing is the identity bypass.
- SSIM rewards leaving small occlusions untouched. For 10% occlusions the untouched input has a higher SSIM than any restored output on most inputs, although its PSNR is about 8 dB lower and the hole is plainly visible; conclusions about occlusion therefore rest on PSNR and visual inspection as well as SSIM. A larger latent, limited skip connections (allowed by the assignment if justified) or a different specialist architecture were not explored.

- Heavy occlusion is irreversible. No model can recover content that is absent; outputs in masked regions are plausible blur, not the original.
- Small searches. Each Optuna study has 8–12 trials on short, sub-sampled runs (Table XVI); three Task 4 trials completed. The selected settings are good, not optimal, and the Task 4 search space did not include alternative GAN objectives.
- Two corrected runs. The first universal model and the first specialists were discarded for the reasons given in Sections IV and V-C; their numbers are reported so that the corrections can be checked. The α range of the final specialist and mixture searches was restricted after looking at the outputs.
- Mini-batch balance in Task 3. Each item of a Task 3 training batch draws its corruption class uniformly, so batches are balanced in expectation but not exactly; the exactly balanced sampler was used for the classifier.
- Evaluation protocol history. The Kaggle evaluation scripts used a single-condition test manifest (3,669 inputs). The full protocol (36,690 inputs) was evaluated afterwards with the deployed ONNX models rather than the PyTorch checkpoints, because the training checkpoints of the final specialists and mixture were not archived from the cloud session; the ONNX models were verified against PyTorch at export time. The training history of the final mixture is transcribed from the console log.
- Single seed, no confidence intervals. Results come from one training run per configuration; we report standard deviations over test inputs, not over training seeds.
- Domain. The restoration models were trained on cats and dogs only; photographs of other subjects (for example faces) are outside their domain and are restored poorly. The cGAN has seen only frontal, near-square FS2K photographs: tall or wide photographs are best cropped, and small faces in wide scenes degrade the sketch.
- Soft sketches and small style 3. The generated sketches carry about 37% of the edge strength of the

real ones, and only 46 test pairs belong to style 3.

- PSNR clipping and infinite PSNR. PSNR is clipped at 60 dB, so averages that include identity-routed images are not directly comparable with literature values.
- Deployment scope. Inference runs on the CPU, which is sufficient (tens of milliseconds per image) but was not load-tested; the webcam path needs browser permission and was tested manually only; the second-round Stitch export of the sketch screen was empty.

XI. Conclusion

A single bottleneck autoencoder restores salt-and-pepper noise well (SSIM $0.382 \rightarrow 0.805$) and fills occlusions plausibly (PSNR $13.6 \rightarrow 21.5$ dB), but it damages images that do not need restoration, such as clean and mildly blurred ones. A classifier with an identity bypass removes that damage (clean SSIM $0.814 \rightarrow 0.996$); the soft, jointly trained mixture does the same (0.991) without any expert becoming inactive. On corrupted inputs the three systems are within about 0.02 SSIM of one another, so the overall improvement is small ($0.772 \rightarrow 0.779 \rightarrow 0.783$) and comes from clean inputs; specialisation itself did not help. The diagnosis of two of our own failures — an under-fitting first model, and specialists that discarded colour because the loss over-weighted SSIM — and the disagreement of SSIM and PSNR on occlusion show that aggregate metrics and Optuna objectives are not sufficient: the outputs must be inspected. The style-conditioned cGAN produces recognisable, style-dependent but soft sketches and leaves room for stronger adversarial or perceptual training. The complete system is deployed as a containerised web application with verified ONNX models, tracked experiments and a one-command start-up.

Appendix A AI-Use Statement

The assignment permits AI assistants provided their use is disclosed and their output is verified. This appendix identifies the tools, the tasks and how the output was tested or corrected. [Author: read this appendix and change anything that does not describe exactly what you did; add the prompts you personally wrote.]

A. Tools and the tasks they were used for

- Claude (web chat, Anthropic). Produced the initial project prompt and the first code base for Tasks 1–3 (data pipeline, corruption code, models, losses, training and Optuna scripts, evaluation and ONNX export scripts).
- Claude Code (coding assistant, Anthropic). Integrated and debugged that code base; wrote the Task 4 cGAN code; wrote the FastAPI back end, the React/Tailwind front end, the Docker set-up, the Kaggle instructions, the verification and screenshot scripts, the report generation scripts and a draft of this report; ran tests and measurements.

- Google Stitch. Generated the interface design from two prompts (Sec. IX); its exported HTML was used as a structural reference, not as the final code.
- Kaggle notebooks (T4 GPU). Compute for all training runs.
- Libraries. PyTorch, Optuna, MLflow, ONNX and ONNX Runtime, scikit-learn, scikit-image, FastAPI, React, Tailwind CSS, Vite, nginx and Docker.

B. How the output was tested or corrected

- Automated tests. Corruption determinism and exact pixel counts, balanced batches, manifest reproducibility, resume-from-checkpoint equivalence (a simulated crash resumes to the same weights), equality of the back end's NumPy corruptions with the training code, 38 back-end API tests, and a 54-check end-to-end test of the running application.
- Numerical verification. Every ONNX export was compared with PyTorch (differences $\leq 2 \times 10^{-6}$), and the published test results were independently re-measured from the ONNX files alone with a different SSIM implementation.
- Corrections made after inspecting results. The first universal autoencoder under-fitted (SSIM 0.545) and was retrained with a corrected search space; the first specialists lost half the colour because Optuna chose an SSIM-heavy loss weight, which a colourfulness measurement revealed, and they were retrained with a restricted range; a second cGAN recipe was measured and rejected; stretching non-square photographs distorted sketches, so a crop option was added; the test manifest was regenerated so that each test image has all ten conditions as the assignment requires; a version-control ignore rule that silently excluded the data package was found and fixed.
- Claims checked against data. Memorisation was examined by comparing training and validation scores; that final models used all training data was confirmed from the MLflow run parameters; that the test set is loaded only by the evaluation scripts was confirmed by searching the code; the layout claim (no scrolling, Run button visible) was measured in a real browser.
- Every number in the report tables and in the text is generated from saved result files by scripts in report/; references were written from the cited papers' bibliographic data and should be checked against the originals.

Appendix B Reproducing the Results

Run the application. Install Docker, then

```
git clone https://github.com/Zaid-fareed/23i-0829_C_GenAi_A1.git
cd 23i-0829_C_GenAi_A1
python scripts/download_models.py
docker compose up --build
```

and open `http://localhost:3000`. The optional experiment browser is started with `docker compose --profile tracking up (port 5000)`.

Retrain. `KAGGLE_GUIDE.md` gives the notebook cells. The order is: `split` and `manifests`; Task 1 (`src.train.task1 optuna, final`); Task 2 classifier and specialists; Task 3; Task 4; then the evaluation and `src.export` scripts. Every final command accepts `--resume` to continue after an interrupted session.

Evaluate. `python -m src.eval.onnx_full_eval` scores the deployed ONNX models on the full 36,690-input manifest; `report/build_figures.py`, `build_tables.py`, `build_qualitative.py` and `build_analysis.py` regenerate every figure, table and number in this report, and `scripts/verify_onnx_results.py` re-measures the test results independently.

Environment. Training: Kaggle GPU T4 with PyTorch; development and inference: CPU. Seeds: 42 throughout.

References

- [1] P. Vincent, H. Larochelle, Y. Bengio, and P.-A. Manzagol, "Extracting and composing robust features with denoising autoencoders," in *Proc. Int. Conf. Machine Learning (ICML)*, 2008, pp. 1096–1103.
- [2] P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio, and P.-A. Manzagol, "Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion," *J. Machine Learning Research*, vol. 11, pp. 3371–3408, 2010.
- [3] K. Zhang, W. Zuo, Y. Chen, D. Meng, and L. Zhang, "Beyond a Gaussian denoiser: Residual learning of deep CNN for image denoising," *IEEE Trans. Image Process.*, vol. 26, no. 7, pp. 3142–3155, 2017.
- [4] X.-J. Mao, C. Shen, and Y.-B. Yang, "Image restoration using very deep convolutional encoder-decoder networks with symmetric skip connections," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.
- [5] D. Pathak, P. Krähenbühl, J. Donahue, T. Darrell, and A. A. Efros, "Context encoders: Feature learning by inpainting," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 2536–2544.
- [6] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Trans. Image Process.*, vol. 13, no. 4, pp. 600–612, 2004.
- [7] H. Zhao, O. Gallo, I. Frosio, and J. Kautz, "Loss functions for image restoration with neural networks," *IEEE Trans. Computational Imaging*, vol. 3, no. 1, pp. 47–57, 2017.
- [8] Y. Blau and T. Michaeli, "The perception-distortion tradeoff," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 6228–6237.
- [9] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton, "Adaptive mixtures of local experts," *Neural Computation*, vol. 3, no. 1, pp. 79–87, 1991.
- [10] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *Int. Conf. Learning Representations (ICLR)*, 2017.
- [11] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *J. Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.
- [12] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2014.
- [13] M. Mirza and S. Osindero, "Conditional generative adversarial nets," *arXiv preprint arXiv:1411.1784*, 2014.
- [14] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, "Image-to-image translation with conditional adversarial networks," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 1125–1134.
- [15] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015, pp. 234–241.
- [16] C. Li and M. Wand, "Precomputed real-time texture synthesis with Markovian generative adversarial networks," in *European Conference on Computer Vision (ECCV)*, 2016, pp. 702–716.
- [17] X. Tang and X. Wang, "Face sketch synthesis and recognition," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2003, pp. 687–694.
- [18] D.-P. Fan, Z. Huang, P. Zheng, H. Liu, X. Qin, and L. V. Gool, "Facial-sketch synthesis: A new challenge," *Machine Intelligence Research*, vol. 19, no. 4, pp. 257–287, 2022.
- [19] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2019, pp. 2623–2631.
- [20] MLflow contributors, "MLflow: an open source platform for the machine learning lifecycle," <https://mlflow.org>.
- [21] ONNX Runtime developers, "ONNX Runtime: cross-platform, high performance ml inferencing and training accelerator," 2021, <https://onnxruntime.ai>.
- [22] S. Ramírez, "FastAPI framework," <https://fastapi.tiangolo.com>.
- [23] D. Merkel, "Docker: Lightweight Linux containers for consistent development and deployment," *Linux Journal*, vol. 2014, no. 239, 2014.
- [24] Google, "Stitch — design user interfaces with AI," <https://stitch.withgoogle.com>.
- [25] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Int. Conf. Learning Representations (ICLR)*, 2019.
- [26] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Int. Conf. Learning Representations (ICLR)*, 2015.
- [27] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in *Int. Conf. Learning Representations (ICLR)*, 2017.
- [28] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. Int. Conf. Machine Learning (ICML)*, 2015, pp. 448–456.
- [29] O. M. Parkhi, A. Vedaldi, A. Zisserman, and C. V. Jawahar, "Cats and dogs," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2012, pp. 3498–3505.

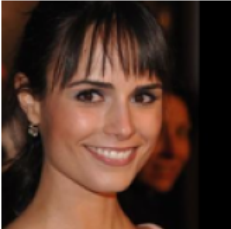
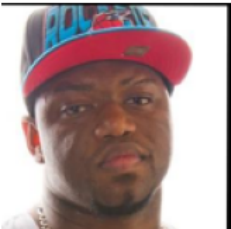
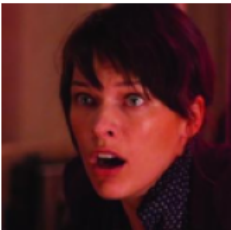
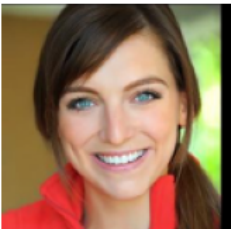
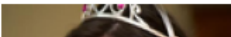
	photo	ground truth	Test examples generated	abs error
style 2				
style 2				
style 2				
style 2				
style 1				
style 1				
style 1				
				



Fig. 22. One photograph rendered in the three styles.

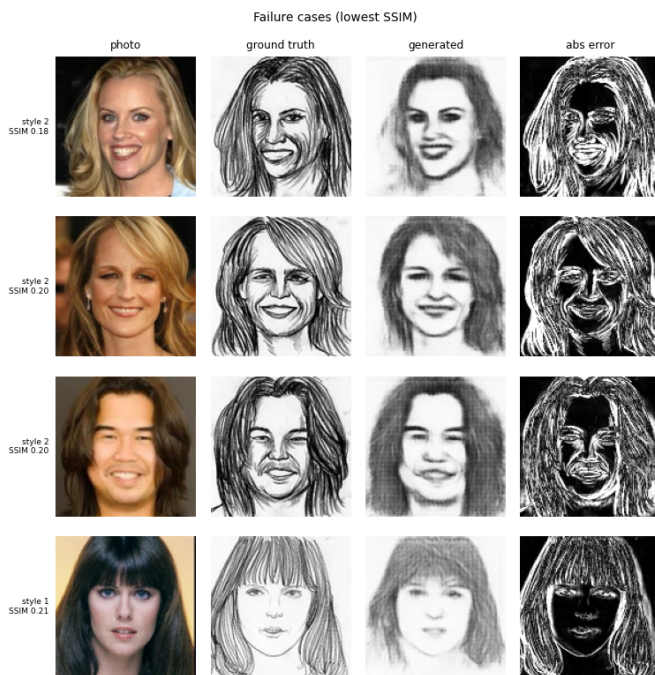


Fig. 23. The four test pairs with the lowest SSIM.

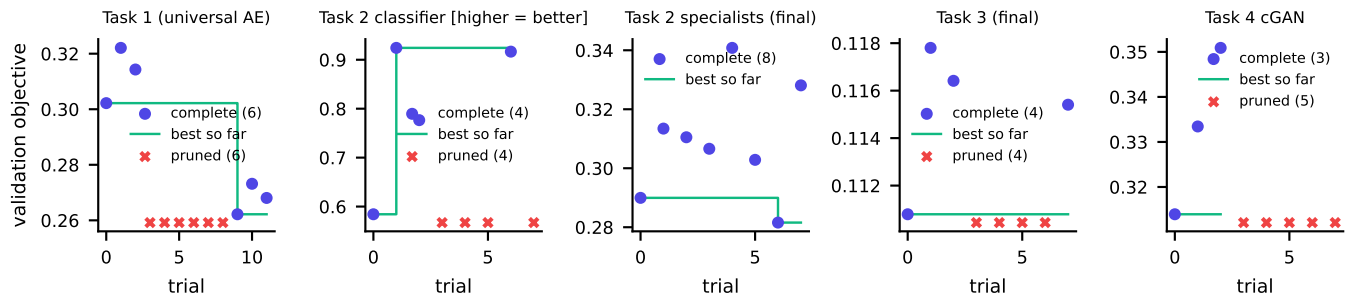


Fig. 24. Optuna studies of the final configurations: objective value of completed trials (dots), best value so far (line) and pruned trials (crosses along the bottom).

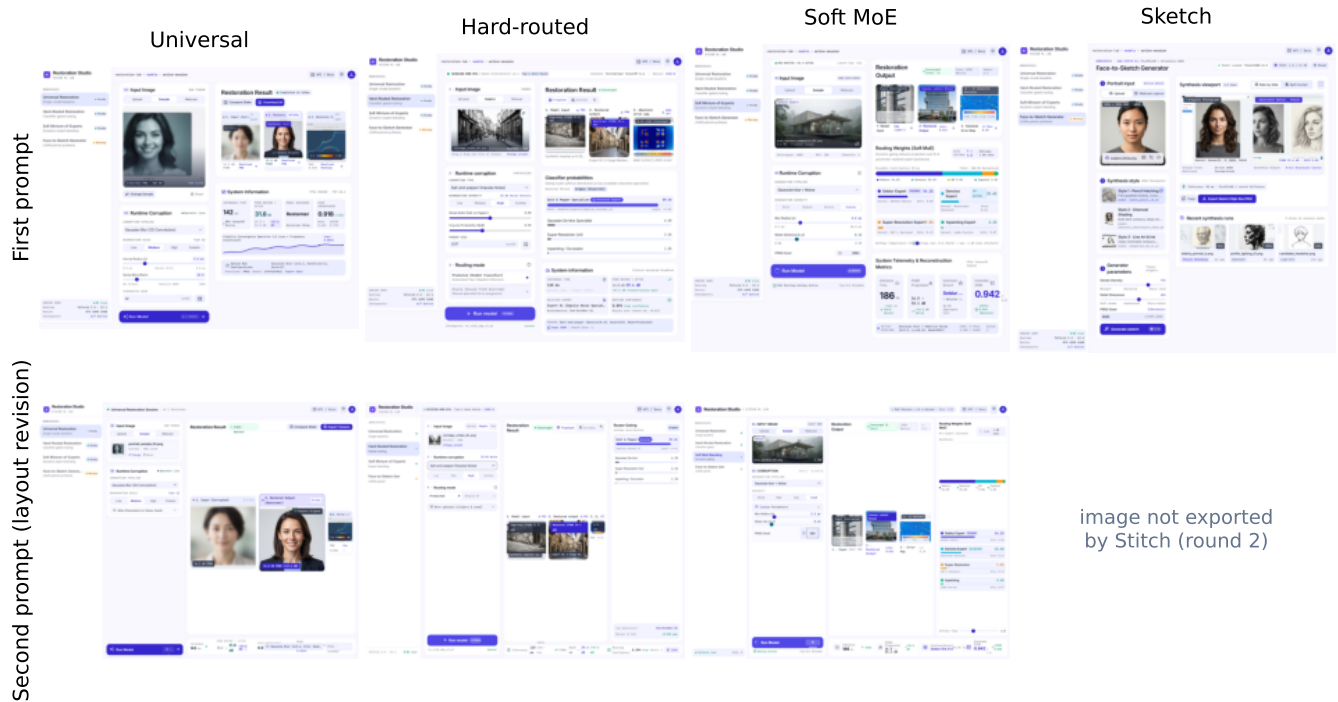


Fig. 25. Google Stitch designs: first prompt (top) and layout revision (bottom).

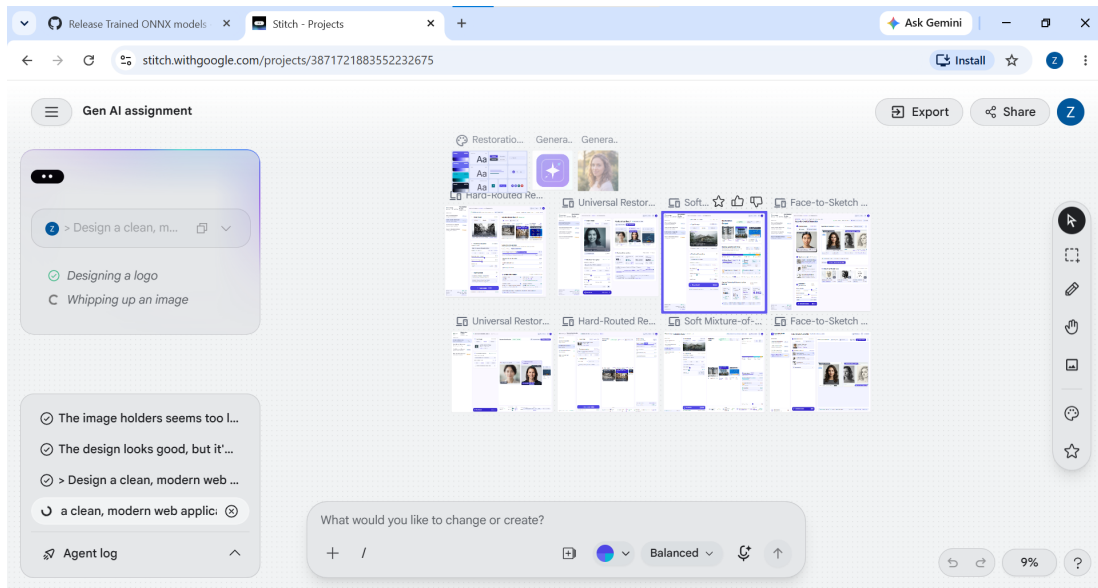


Fig. 26. The Google Stitch project page: the screens of both design rounds on the canvas and the list of prompts on the left.

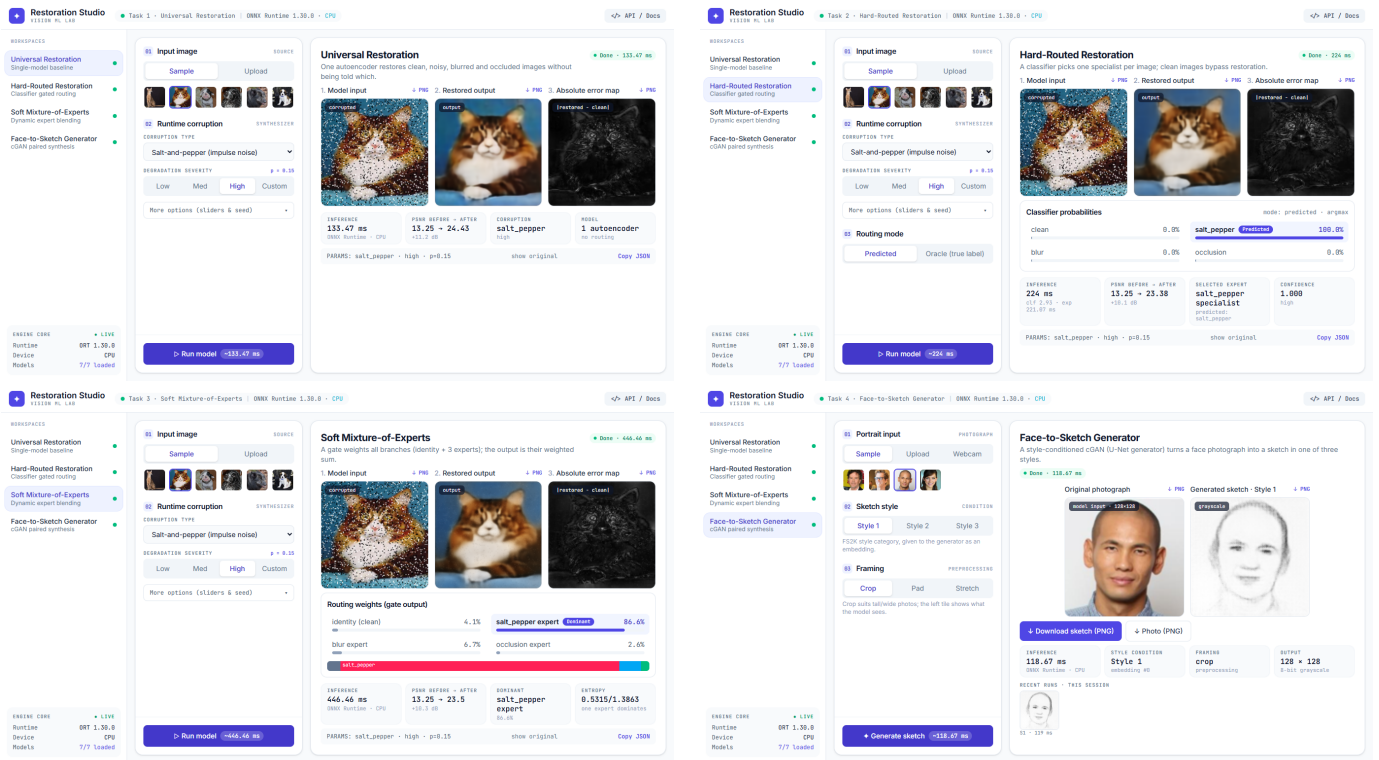


Fig. 27. Screenshots of the deployed application (1366x768): universal restoration, hard-routed restoration, soft mixture-of-experts restoration and the face-to-sketch generator, each after a run on a built-in sample.

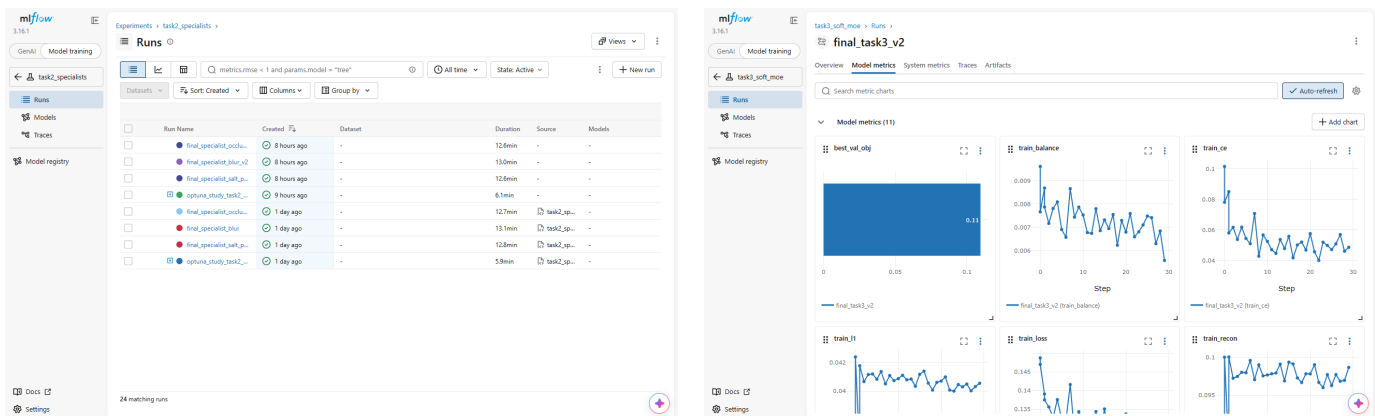


Fig. 28. MLflow interface: the experiments with their runs (left) and the metrics of one training run (right).